RAMACHANDRA COLLEGE OF ENGINEERING

(Autonomous)

UNIT-01

UNIT - I 8086/8088 MICROPROCESSORS :

A.RAMADEVI

ASSISTANT PROFESSOR

Course Objectives:

1 To understand the internal architecture, functional components, operational modes,
instruction formats, and programming features of 8086/8088 microprocessors.

Course Outcomes:

At the end of the course, students will be able to
CO 1 Develop programs for different addressing modes.

SYLLUBUS

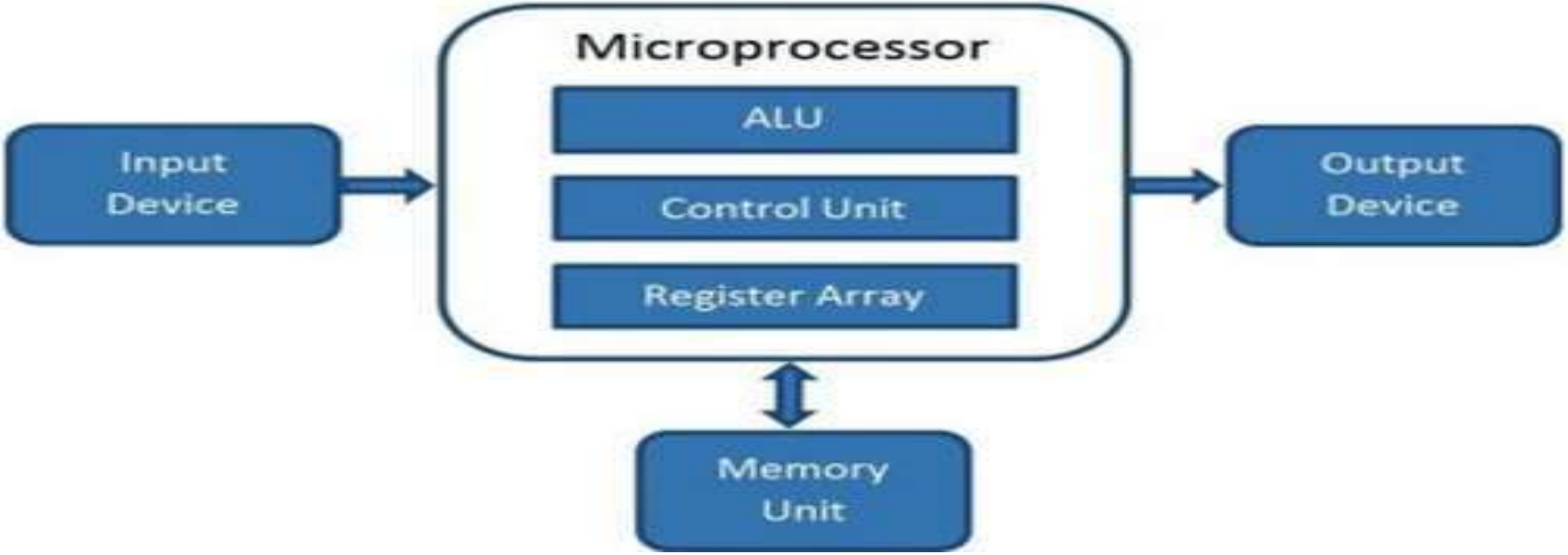
Register organization of 8086, Architecture, signal capability, special purpose activities, Minimum mode, maximum mode of 8086 system and timings, machine language instruction
formats, addressing mode of 8086, instruction set of 8086,assembler directives and operators

UNIT - I 8086/8088 MICROPROCESSORS :

- Register organization of 8086,
- Architecture,
- signal capability, special purpose activities,
- Minimum mode, maximum mode of 8086 system and timings,
- machine language instructionformats
- addressing mode of 8086,
- instruction set of 8086
- ,assembler directives and operators).

What is a Microprocessor?

- ✓ Computer's Central Processing Unit (CPU) built on a **single Integrated Circuit (IC)** is called a **microprocessor**.
- ✓ A digital computer with one microprocessor which acts as a CPU is called microcomputer.
- ✓ It is a **programmable, multipurpose, clock -driven, register-based electronic device** that **reads binary instructions from a storage device called memory, accepts binary data as input and processes data according to those instructions and provides results as output.**
- ✓ A microprocessor is an electronic component that is used  It is a central processing unit on a single integrated circuit chip containing millions of very small components including transistors, resistors, and diodes that work together.



Working of Microprocessor:

- ✓ A microprocessor consists of an ALU, control unit and register array. Where **ALU** performs arithmetic and logical operations on the data received from an input device or memory.
- ✓ Control unit controls the instructions and flow of data within the computer. And, **register array** consists of registers which access the data very fast.

CPU: The Central Processing Unit or CPU controls the operation of the computer. In a microcomputer the CPU is a microprocessor. The CPU or Microprocessor fetches binary ceded instructions from the memory, decodes the instructions into a series of simple actions, and carries out these actions in sequence of steps.

Address bus: The address bus consists of 16, 20, 24, or 32… parallel signal lines. These address lines are used to send a address of the memory location or a device address from the microprocessor unit to the memory or the peripheral. The address bus always unidirectional. Address always goes out of the microprocessor. The number of memory locations that the microprocessor can address is determined by the number of address lines. If the microprocessor has N address lines, then it can directly address 2^N memory locations. For ex: a microprocessor with 16 address lines can address 2^{16} or 65,536 memory locations.

Data bus: The data bus consists of 8,16,or 32…parallel lines .A group of lines used to transfer a data between the microprocessor and peripherals(or memory).The data bus is always bi-directional.

Control bus: The control bus consists of 4 to 10 parallel signal lines. The microprocessor sends out signals on the control bus to enable the outputs of addressed memory devices or port devices. Typical control bus signals are Memory Read, Memory Write, I/O Read, and I/O Write.

SOME IMPORTANT TERMS:

Microprocessor: It is a semiconductor device which is manufactured by using LSI or VLSI technology, which includes ALU, Control unit and a group of Registers in a single Integrated circuit.

Microcontroller: It is a device that includes microprocessor, memory, and I/O signal lines on a single chip, fabricated using VLSI technology.

Microcomputer: A digital computer having a microprocessor as its Central Processing Unit is called Microcomputer. So, a microprocessor combined with memory, an input device and an output device forms a microcomputer.

Bus: A group of wires or lines used to transfer bits between the microprocessor and other components of the computer system. Or a path used to carry signals, such as connection between memory and the microprocessor in a digital computer.

Hardware: The physical devices and circuitry of the computer is called Hardware.

Software: The programs written for the computer is referred to as software.

Firmware: The programs stored in ROMs or in other devices which permanently keep their stored information are referred as Firmware.

- ✓ In general the width of the data bus is equal to the bit capacity of the microprocessor.
- ✓ In general the internal architecture of the microprocessor depends on the bit capacity of the microprocessor.
  - **Bit**-a binary digit.0 or 1
  - **Nibble**-a group of four bits
  - **Byte**-a group of eight bits
  - **Word**-a group of 16 bits or a group of bits the computer recognizes and processes at a time.
  - **Double word**-a group of 32 bits.

Instruction: A command in binary that is recognized and executed by the computer to accomplish a task. Some instructions are designed with one word, and some require multiple words.

Mnemonic: A combination of letters to suggest the operation of an instruction.

Program: A set of instructions written in specific sequence for the computer to accomplish a given task.

Machine language: The binary medium of communication with a computer through a designed set of instructions specific to each computer.

Assembly language: A medium of communication with a computer in which programs are written in mnemonics.

Low-level language: A medium of communication that is machine-dependant or specific to a given computer.

The machine and the assembly languages of a computer are considered low level languages. Programs written in these languages are not transferable to different types of machines.

High-level language: A medium of communication that is independent of a given computer. Programs are written in English like words, and they can be executed on a machine using translator (a compiler or interpreter).

Source code: A program written either in mnemonics of as assembly language or in English like statements of high level language (before it is assembled or compiled).

Assembler: A computer program that translates an assembly language program from mnemonics to the binary machine code of a computer.

Compiler: A program that translates English-like words of a high level language into the machine language of a computer.

Interpreter: A program that translates the English –like statements of a high level language into the machine language of a computer.

Operating System: A set of programs that manages interaction between hardware and software.

ASCII: American Standard Code for Information Interchange. This is 7-bit alphanumeric code with 128 combinations.



APPLICATIONS OF MICROPROCESSORS:

Microprocessors are used in :

1.Calculators.

2.Accounting system.

3.Games machine.

4.Complex Industrial Controllers.

5Traffic light Control.

6.Data acquisition systems.

2.EVOLUTION OF MICROPROCESSORS

First Generation (4 - bit Microprocessors)

- ✓ The first generation microprocessors were introduced in the year 1971-1972 by Intel Corporation. It was named **Intel 4004** since it was a 4-bit processor.
- ✓ It was a processor on a single chip. It could perform simple arithmetic and logical operations such as addition, subtraction, Boolean OR and Boolean AND.
- ✓ It had a control unit capable of performing control functions like fetching an instruction from storage memory, decoding it, and then generating control pulses to execute it.

Second Generation (8 - bit Microprocessor)

- ✓ The second generation microprocessors were introduced in 1973 again by Intel.
- ✓ It was a first 8 - bit microprocessor which could perform arithmetic and logic operations on 8-bit words.
- ✓ It was Intel 8008, and another improved version was Intel 8088.

Third Generation (16 - bit Microprocessor)

- ✓ The third generation microprocessors, introduced in 1978 were represented by Intel's 8086, Zilog Z800 and 80286, which were 16 - bit processors with a performance like minicomputers.

Fourth Generation (32 - bit Microprocessors)

- ✓ Several different companies introduced the 32-bit microprocessors in around 1980, but the most popular one is the **Intel 80386**.

Fifth Generation (64 - bit Microprocessors)

- ✓ From 1995 to now we are in the fifth generation.
- ✓ After 80856, Intel came out with a new processor namely Pentium processor followed by **Pentium Pro CPU**, which allows multiple CPUs in a single system to achieve multiprocessing.
- ✓ Other improved 64-bit processors are **Celeron, Dual, Quad, Octa Core processors**.

Functions of Microprocessor:

1. Controlling all other parts of the machine and sending timing signals.

2. Transferring data between memory and I/O devices.

3. Fetching data and instructions from memory.

4. Decoding instruction.

5. Performing arithmetical and logical operations.

6. Executing programs stored in memory.

7. Performing communication among the I/O devices etc.

Features of Microprocessor

- **Low Cost** - Due to integrated circuit technology microprocessors are available at very low cost. It will reduce the cost of a computer system.
- **High Speed** - Due to the technology involved in it, the microprocessor can work at very high speed. It can execute millions of instructions per second.
- **Small Size** - A microprocessor is fabricated in a very less footprint due to very large scale and ultra large scale integration technology. Because of this, the size of the computer system is reduced.
- **Versatile** - The same chip can be used for several applications, therefore, microprocessors are versatile.
- **Low Power Consumption** - Microprocessors are using metal oxide semiconductor technology, which consumes less power.
- **Less Heat Generation** - Microprocessors uses semiconductor technology which will not emit much heat as compared to vacuum tube devices.
- **Reliable** - Since microprocessors use semiconductor technology, therefore, the failure rate is very less. Hence it is very reliable.
- **Portable** - Due to the small size and low power consumption microprocessors are portable.

.REGISTER ORGANIZATION OF 8086 MICROPROCESSOR:

Register Organization

a) General purpose data registers

b) Segment Registers

c) Index and Point Registers



Figure: Register organization of 8086 Microprocessor

a) General purpose data registers

Accumulator register (AX) **consists of two 8-bit registers AL(**Low-order byte) **and AH(**High-order byte), which **can be combined together and used as a 16-bit register AX.** AL in this case contains and AH contains, **It holds operands and results during operation**

Base register (BX) **consists of two 8-bit registers BL(**Low-order byte) **and BH(**High-order byte), which **can be combined together and used as a 16-bit register BX .it holds the offset address in indirect addressing mode**

Count register (CX) **consists of two 8-bit registers CL(**Low-order byte) **and CH(**High-order byte), which **can be combined together and used as a 16-bit register CX. It holds count for instruction.**

Data register (DX) **consists of two 8-bit registers DL(**Low-order byte) **and DH(**High-order byte), which **can be combined together and used as a 16-bit register DX. It used with AX to holds 32bit values during MULTIPLICATION AND DIVISION operation.**

b) Segment Registers

Code segment (CS) is a 16-bit register containing address of code segment(64KB) of memory, **where the executable program is stored.**

Stack segment (SS) is a 16-bit register containing address of stack segment(64KB) of memory. which is used to store stack data i.e., **subroutines or functions**

Data segment (DS) is a 16-bit register containing address of data segment(64KB) of memory, **where data is stored**.

Extra segment (ES) is a 16-bit register containing address of extra segment(64KB) of memory, **where data is stored.**

c) Index and Point Registers

Instruction Pointer (IP) is a 16-bit register. The IP register holds the Offset address of the next instruction to be fetched from **Code segment.**

Stack Pointer (SP) is a 16-bit register. The SP register holds the Offset address of TOP of **STACK SEGMENT.**

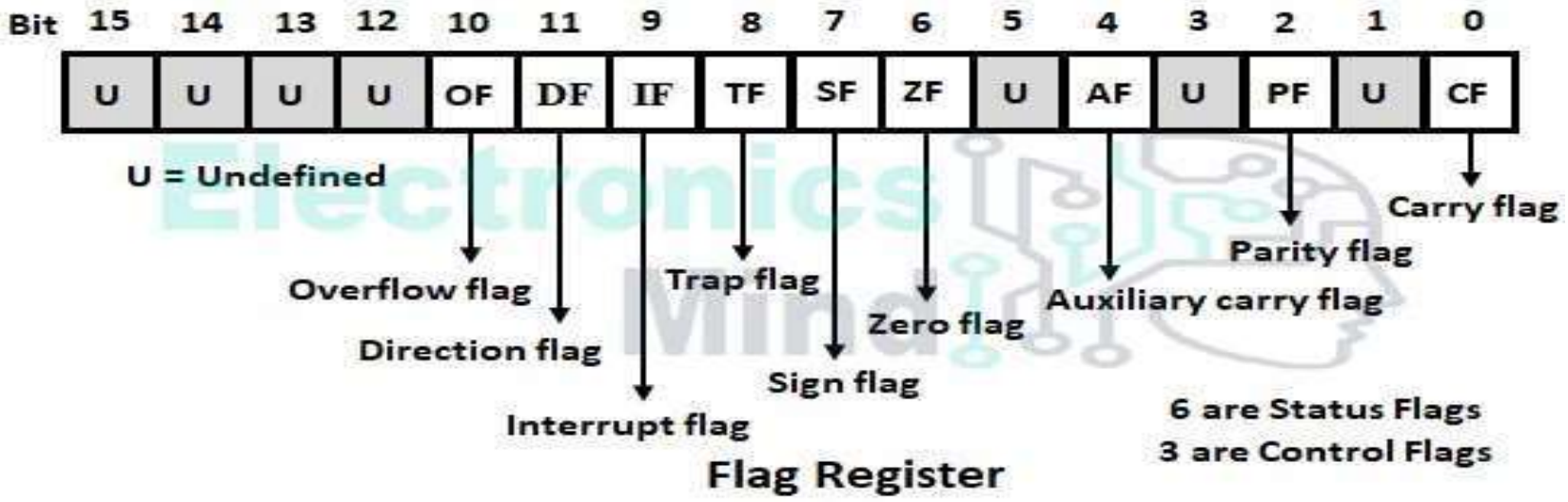
Base Pointer (BP) is a 16-bit register. The BP register holds the Offset address of any random location OF **STACK SEGMENT.**

Source Index (SI) is a 16-bit register. The SI register holds the Offset address of **DATA SEGMENT**

Destination Index (DI) is a 16-bit register. The SI register holds the Offset address of **EXTRA SEGMENT**

d) Flag Registers

8086 has 16-bit flag register, and there are 9 valid flag bits i.e., 6- status flags and 3-control flags. The format of flag register is like below.



CF:CARRY FLAG

CF = 1

```
   1 1
  1011 1100
+ 1001 0001
-----------
1 0100 1101
```

CF = 0

```
   1 1
  1011 1100
+ 0001 0001
-----------
  1100 1101
```

PF:PARITY FLAG

PF = 1

```
   1 1
  1011 1100
+ 1001 0001
-----------
1 0100 1101
```

PF = 0

```
   1 1
  1011 1100
+ 0001 0001
-----------
  1100 1101
```

If PF = 1 ; it means it is even parity in the result (there are even numbers of 1's).

If PF = 0 ; it means it is odd parity.

AF:AUXILARY FLAG



If AF = 1 ; there is a carry out from lower nibble.
If AF = 0 ;no carry out of lower nibble.

✓ZF:ZERO FLAG

- ✓ **This is zero flag. Whenever the output is 0 this flag is 1.**
- ✓ **If ZF = 1 ; output is zero.**
- ✓ **If ZF = 0 ; output is non zero.**

SF:SF stands for Sign Flag.

MSB = 1 , then it is +ve number

MSB = 0 , then it is -ve number. This is the reverse with the original case.

```
  1 1
  1011 1100
+ 1001 0001
-------------
1 0100 1101
```

```
  1 1
  1011 1100
+ 0001 0001
-------------
  1100 1101
```

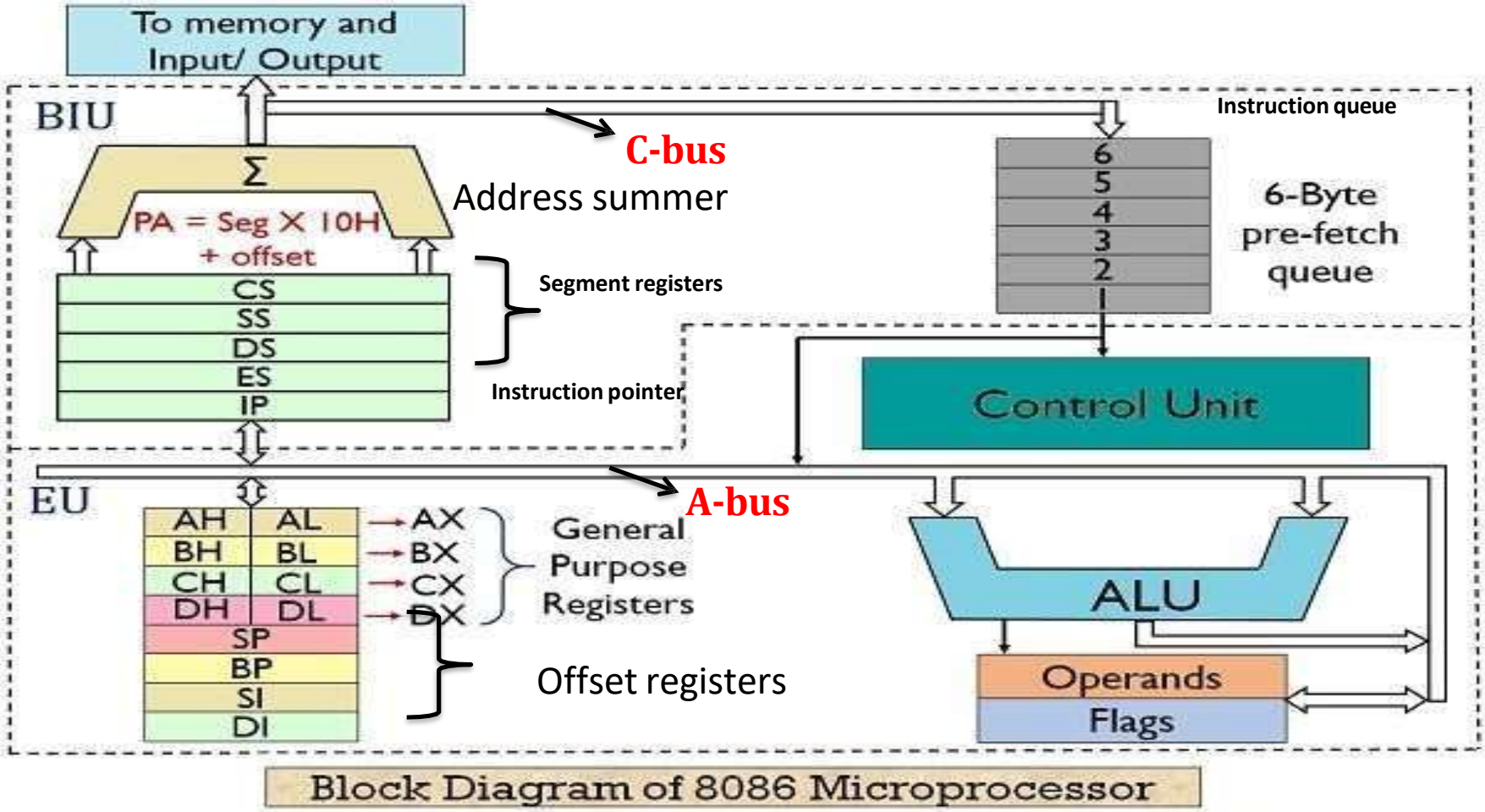
OF: The **overflow flag** is set to 1 when the result of a signed operation is too large to fit.

Flag Bit	Function
SF	**Sign Flag**-After any operation if the MSB is 1, then it indicates that the number is negative. And this flag is set to 1
ZF	**Zero Flag-**If the total register is zero, then only the Z flag is set
AF	**Auxiliary Flag**-When some arithmetic operations generates carry after the lower half and sends it to upper half, the AC will be 1
PF	This is even **parity flag**. When result has even number of 1, it will be set to 1, otherwise 0 for odd number of 1s
CF	This is **carry flag**. If some operations are generating carry after the operation this flag is set to 1
OF	The **overflow flag** is set to 1 when the result of a signed operation is too large to fit.

Flag Bit	Function
DF	**This is directional flag.** This is used in string related operations. D = 1, then the string will be accessed from higher memory address to lower memory address, and if D = 0, it will do the reverse.
IF	**This is interrupt flag**. If I = 1, then MPU will recognize the interrupts from peripherals. For I = 0, the interrupts will be ignored
TF	**This Trap flag** is used for on-chip debugging. When T = 1, it will work in a single step mode. After each instruction, one internal interrupt is generated. It helps to execute some program instruction by instruction.T=0 NO EXECUTION IS PERFORMED.

3.ARCHITECTURE OF 8086 MICROPROCESSOR

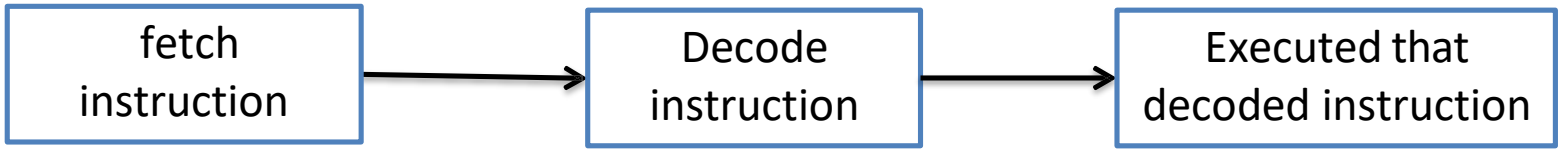
https://www.youtube.com/watch?app=desktop&v=OvLRBSQ0Ja0


To memory and Input/ Output
BIU
Σ
PA = Seg X 10H + offset
CS
SS
DS
ES
IP
C-bus
Address summer
Segment registers
Instruction pointer
Instruction queue
6
5
4
3
2
1
6-Byte pre-fetch queue
Control Unit
EU
A-bus
AH
AL
BH
BL
CH
CL
DH
DL
AX
BX
CX
DX
General Purpose Registers
SP
BP
SI
DI
Offset registers
ALU
Operands
Flags
Block Diagram of 8086 Microprocessor
Electronics Desk

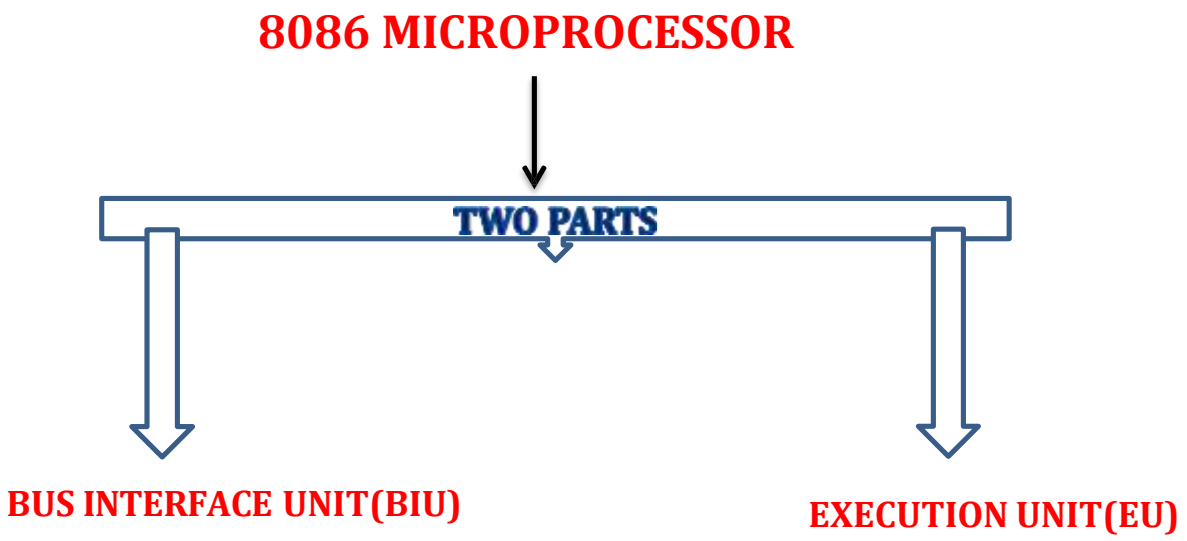
- ✓ It is a 16 bit processor i.e., it process and operates 16 bit data at a time.
- ✓ It is capable of working on 16 bits of information in one cycle or at a time.
- ✓ 8086 MICROPROCESSOR had16 bit data bus and size of the address bus is 20 bit

i.e., 2^ 20=1048576=1MB

- ✓ 8085 microprocessor fetch the instruction from memory and get inside the processor and then to execute that instruction then after go and fetch next instruction that drawback is over-come in 8086 microprocessor
- ✓ In 8086 microprocessor fetch the instruction from memory and get inside the processor and while executes that instruction it will be fetching next instruction side by side it is done by using BIU & EU.**In any processor to do**

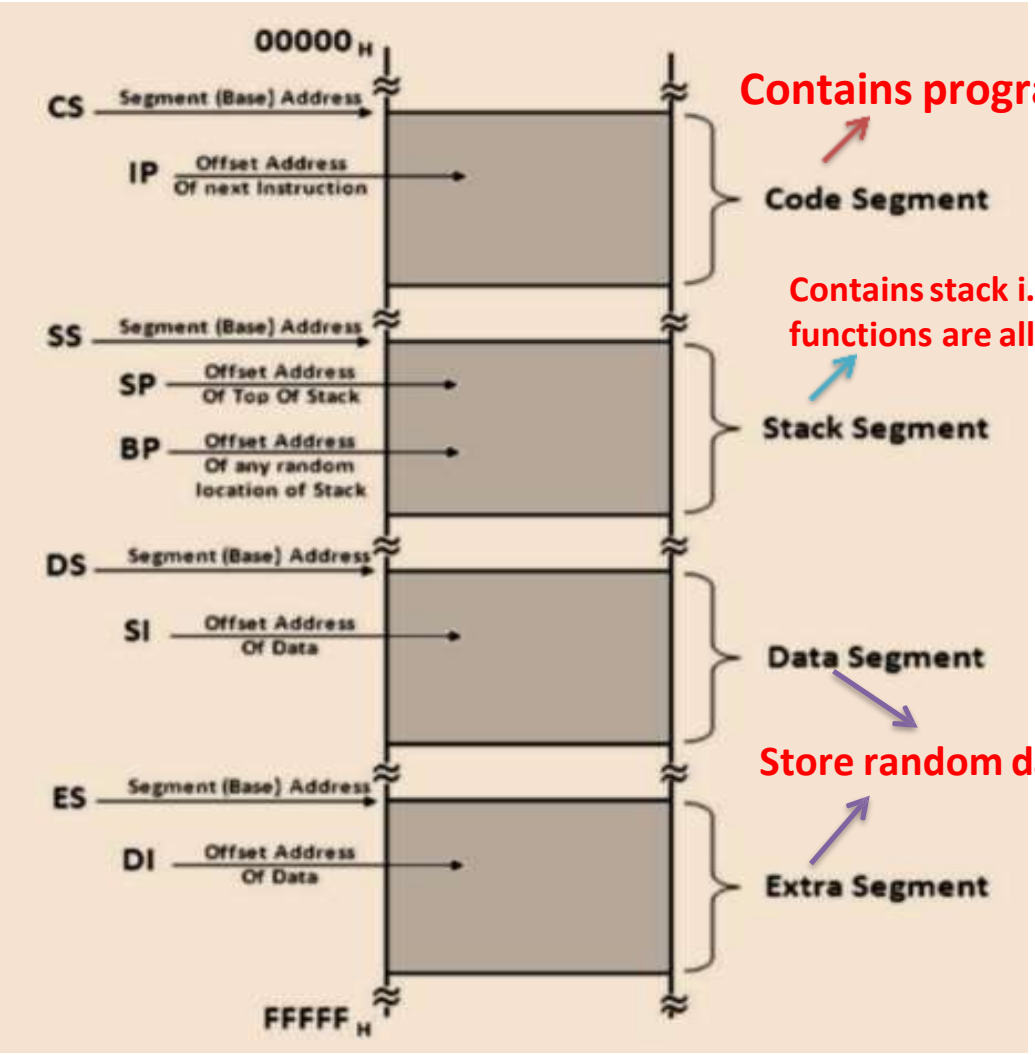


- ✓ Who needs to calculate physical address ?
- ✓ When ever EU executes an instruction at that time BIU wants to fetch next instruction from memory that's why BIU need to calculate physical address.
- ✓



Bus Interface Unit of Microprocessor 8086

- ✓ BIU is used to fetch the instructions from memory
- ✓ BIU is used to calculate Physical Address
- ✓ BIU is used to manage instruction Queue
- ✓ Here, every instruction address it changes offset address not segment address
- ✓ Here, Code Segment(CS) GIVES STARTING ADDRESS
- ✓ Here, Instruction Pointer(IP) always goes on increases give address i.e., offset address to next instruction.



- ✓ Memory is divided into 4 parts is called as Segments.
- ✓ Segments are presented in memory
- ✓ Segment registers are presented in processor they contain address of all the segments
- ✓ Processor calculate real address or physical address

- ✓ **Why we calculate Physical address ?**
- ✓ For example do you know every location of your mobile phone or laptop it is very typical because we know like one folder inside one file that is virtual address it is not real address.
- ✓ Every location has its own address or physical address.
- ✓ To calculate Physical address by using the following formula
- ✓ **PHYSICAL ADDRESS=SEGMENT REGISTER ADDRESS*10H+OFFSET ADDRESS**

Assume:

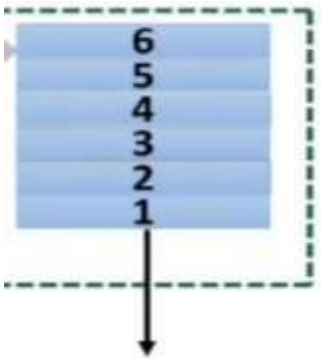
- ✓ **SEGMENT REGISTER ADDRESS=1000H**
- ✓ **OFFSET ADDRESS=12345H**
- ✓ **SO,PHYSICAL ADDRESS=1000H*10H+1234H=12345H**

- ✓ Both CS,IP are used to fetch next instructions from memory through data bus.
- ✓ **Buses are three types:**
- ✓ **Address bus** gave address and **code bus** gave read signal to fetch the instruction from memory through **Data Bus.**
- ✓ **BIU Fetching instructions in advance is called as pre-fetching from memory comes into Instruction Queue through data bus.**
- ✓ **Instruction Queue it woks on First In First Out(FIFO).**
- ✓ **So, In instruction queue , the instructions are move to Execution Unit and these instructions are executed continuous.**

INSTRUCTION QUEUE:

- ✓ In **instruction queue**, it is a 6-byte instruction queue and it is used to store pre fetching instruction from memory through data bus
- ✓ Number of bytes in queue is 06 but each instruction size is not a single byte or all the instructions are not having same size and maximum size of the instruction is 6-Bytes.

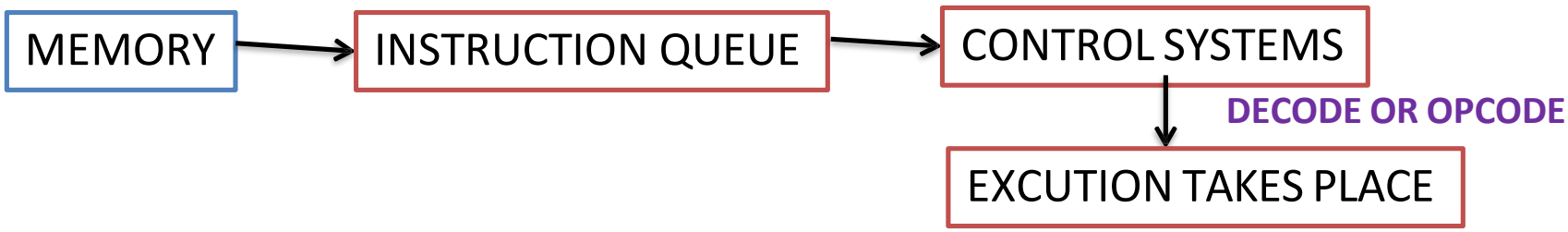
Instruction queue



- ✓ Suppose one byte instruction is executed .so one byte is empty in queue but BIU not do anything.
- ✓ Now two bytes are empty BIU fetches instructions from memory and filled it or pre fetching instructions from memory through data bus.
- ✓ Reason is 8086 microprocessor is a 16 bit processor.

Execution Unit of Microprocessor 8086

- The EU performs following functions:
  - It picks up the instruction from the instruction queue of BIU.
  - It decode the instructions and then executes the instruction.
  - It updates the status of flag register.

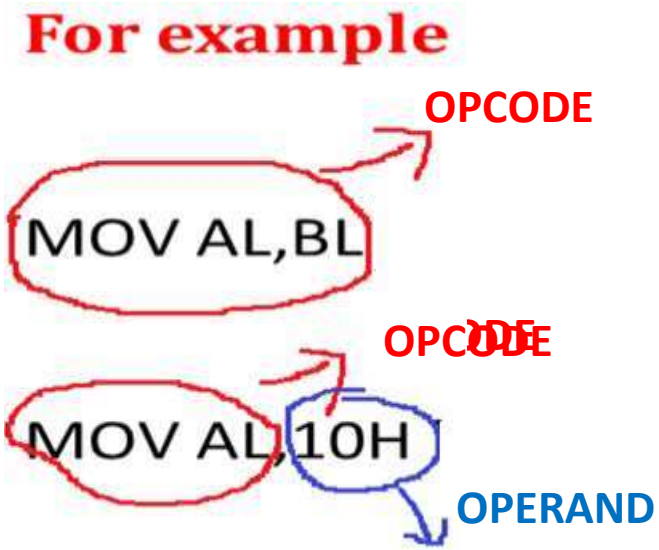


CONTROL SYSTEM/UNIT:

For example

MOV AL,BL here,BL comes to AL

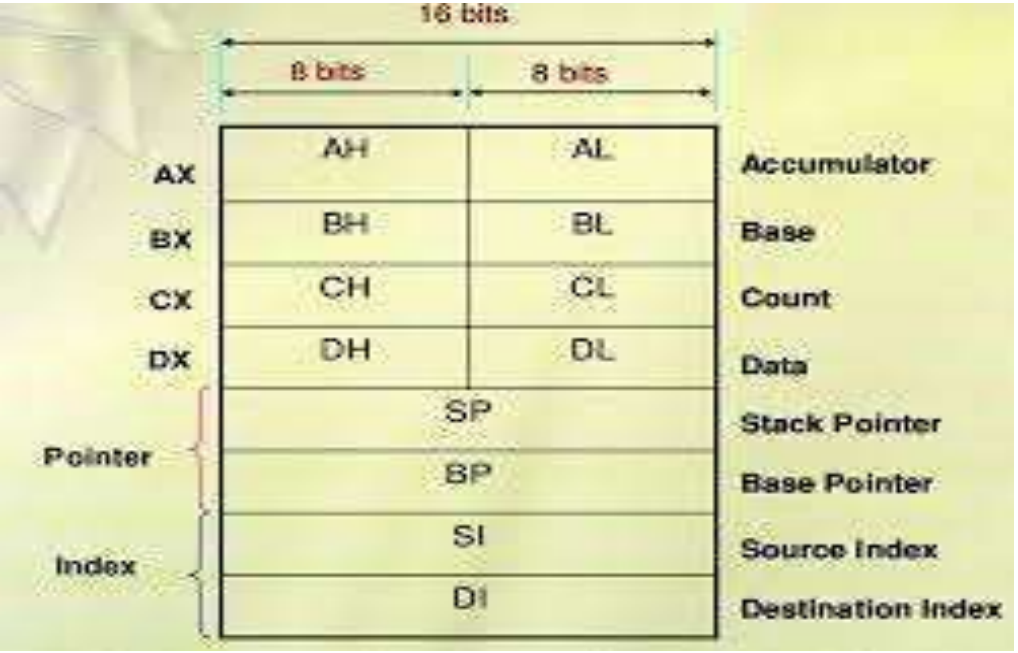
MOV AL,10H here AL gets 10H



- ✓ NUMBER IS NOT INCLUDED IN OPCODE
- ✓ IT SPECIFIES FOR REGISTERS AND OPERATION

DECODE/OPCODE: it is used to convert ASSEMBLY LANGUAGE TO BINARY LANGUAGE. Microprocessor is a middle level language i.e., ASSEMBLY LANGUAGE but processor couldn't understand that language because it knows machine language /binary language or 0's & 1's ONLY

GENERAL PURPOSE REGISTERS:



EXAMPLE:

- ✓ MOV AL,34H means AL ← 34H
- ✓ MOV AH,12H means AH ← 12H
- ✓ MOV AX,1234H ← means AH 34H AND AL ← 12H

EXAMPLE 1:

- ✓ **MOV BL,CL means BL ← CL**
- ✓ **MOV BH,CH means BH ← CH**
- ✓ **MOV CX,AX means CX ← AX**

EXAMPLE 2:

- ✓ **MOV BL,04H**
- ✓ **MOV CL,O5H**
- ✓ **ADD BL,CL**

INFORM TO GENERAL PURPOSE REGISTERS BY CONTROL SYSTEM

ALU IS USED TO PERFORM ARITHETIC AND LOGIC OPERATION

OPERAND REGISTERS :

- ✓ It is a temporary register and it stores temporary values by processor.

Example:

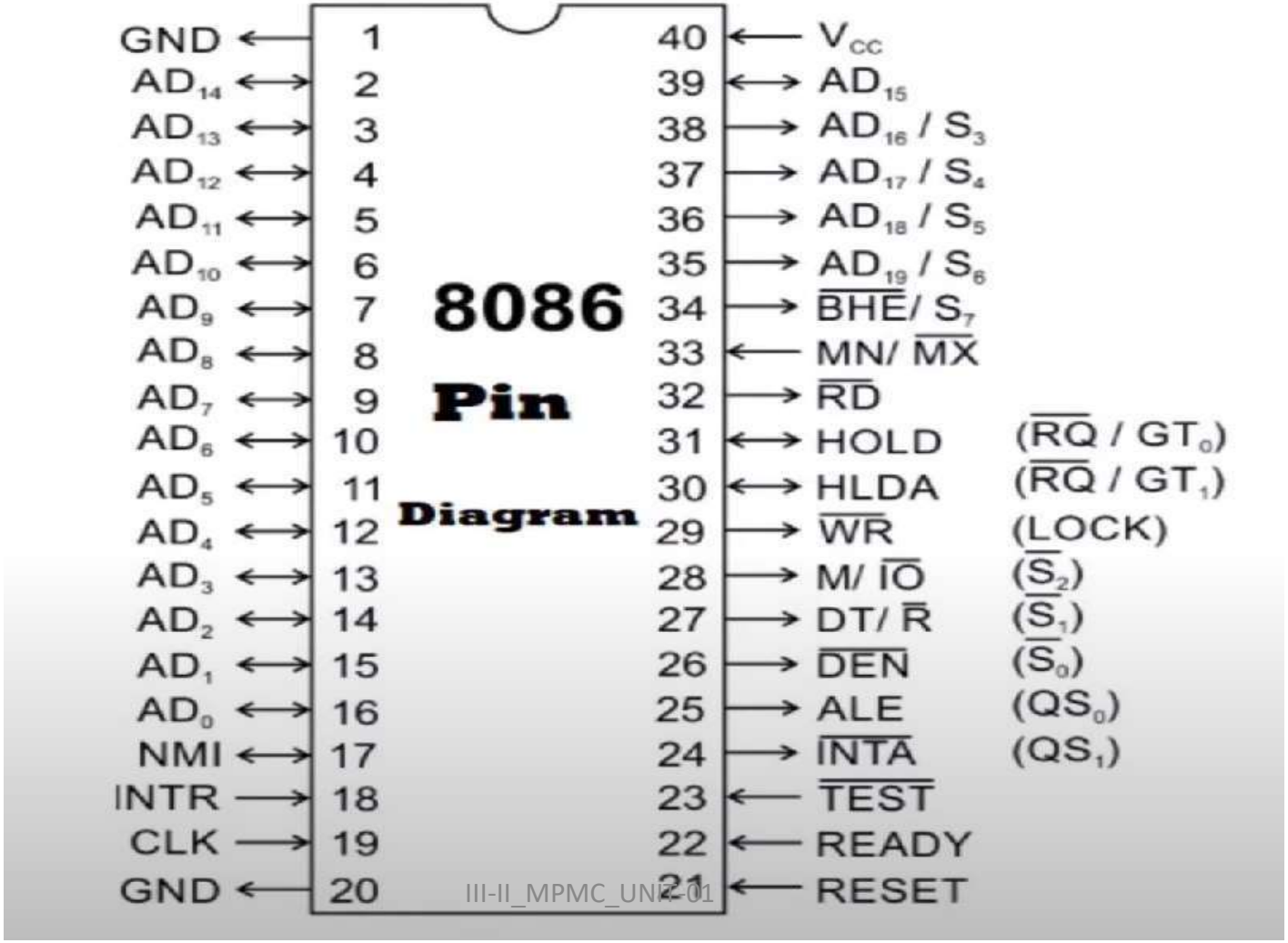
XCHG BX,CX

here, exchange of two values by using temporary value that is stored into operand register.

FLAGS:

- ✓ Each flag gives status of current register.

6.PIN DIAGRAM OF 8086 MICROPROCESSOR


8086
Pin
Diagram
GND 1
AD14 2
AD13 3
AD12 4
AD11 5
AD10 6
AD9 7
AD8 8
AD7 9
AD6 10
AD5 11
AD4 12
AD3 13
AD2 14
AD1 15
AD0 16
NMI 17
INTR 18
CLK 19
GND 20
40 VCC
39 AD15
38 AD16 / S3
37 AD17 / S4
36 AD18 / S5
35 AD19 / S6
34 BHE/ S7
33 MN/ MX
32 RD
31 HOLD (RQ / GT0)
30 HLDA (RQ / GT1)
29 WR (LOCK)
28 M/ IO (S2)
27 DT/ R (S1)
26 DEN (S0)
25 ALE (QS0)
24 INTA (QS1)
23 TEST
22 READY
21 RESET

Microprocessor 8086 PIN Diagram



- **Address Data Bus – [AD0-AD15]**
  - 8086 has 20 lines for Address bus and 16 Data Lines.
  - Here, AD0 – AD15 are time multiplexed Address Data Lines and can be separated by ALE terminal.
  - If ALE = 1 then it carries Address [A0-A15] and If ALE = 0 then it carries data [D0-D15].
- **Address Status Lines – [A19/S6-A16/S3]**
  - 8086 has 4 time multiplexed Address status lines A19/S6 – A16/S3 and can be separated by ALE terminal.
  - If ALE = 1, then it carries Address [A19-A16] and If ALE = 0 then it carries Status signals [S6-S3].
- **Status signals [S7/S3]**
  - S3 & S4 indicates which segment is accessed by 8086 during current bus cycle.
  - S5 reflects IF flag register.
  - S6 is always zero.
  - S7 is always one.

S4	S3	Segment Accessed
0	0	ES
0	1	SS
1	0	CS
1	1	DS

Microprocessor 8086 PIN Diagram



- **Bus High Enable – [$\overline{BHE}$/S7]**
  - This line is used to enable D15-D8 of D15-D0.
  - It is used by Microprocessor for memory banking.
- **Non Maskable Interrupt – [NMI]**
  - This line is used give highest priority interrupt to Microprocessor 8086.
  - It can not be disabled by the software.
  - It is positive edge triggered interrupt.
  - When it occurs, Type 2 Interrupt occurs in the 8086.
- **Interrupt Request – [INTR]**
  - This is level triggered hardware interrupt.
  - It depends of status of Interrupt Flag.
  - If IF = 1, INTR = 1, the 8086 gets interrupted.
  - If IF = 0, INTR = 1, then INTR is disabled.
- **Interrupt Acknowledgment – [$\overline{INTA}$]**
  - Along with Interrupt request on INTR, Microprocessor gives interrupt acknowledgment to peripherals.

Microprocessor 8086 PIN Diagram

AD0-AD15

A19/S6-A16/S3

$\overline{BHE}$/S7

ALE

NMI

INTR

$\overline{INTA}$

CLK

MN/$\overline{MX}$

Ready

$\overline{RD}$

System Clock – [CLK]

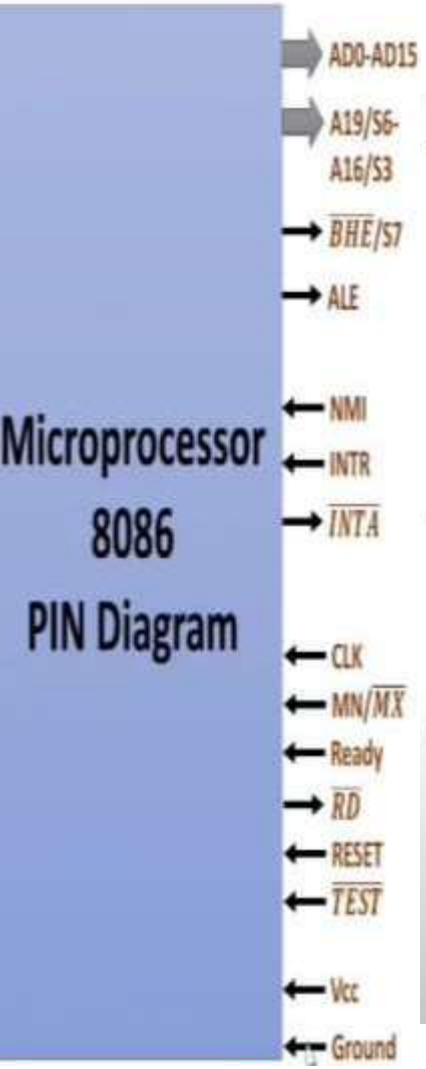
- Clock is given to 8086 for internal timings.
- Clock with 8086 is 5MHz, 8MHz & 10Mhz.

Minimum & Maximum Mode [MN/$\overline{MX}$]

- For Minimum Mode connected with Vcc.
- With Minimum Mode 8086 will work as single microprocessor.
- For Maximum Mode connected with ground.
- With Maximum mode 8086 will works with multiple coprocessor.

Ready Signal [READY]

- This input is used to insert the wait state into timing cycle of microprocessor 8086.
- If the ready pin is at logic 1, it has no effect on the operation.
- If it is logic 0, 8086 enters into the wait state like idle.
- It is used to synchronize slow peripheral devices.



- **System Reset – [RESET]**
  - This input will reset microprocessor 8086.
  - If it held logic 1 for minimum of 4 clock cycles then microprocessor will get RESET.
  - After RESET of 8086, CS and IP initialized to FFFFH and 0000H, so physical address will be FFFF0H.
  - Remaining registers initialized to zero.
- **Test [$\overline{TEST}$]**
  - It is used for synchronization.
  - When this input is logic 0, 8086 executes WAIT instruction.
- **Power Supply [Vcc & Ground]**
  - 8086 is given with 5 Volt DC supply.
  - Allowed variation is 10%.
  - 8086 has two ground pins.
  - This two ground pins are there to have less power dissipation.

MINIMUM MODE(MN):IT IS ACTIVE HIGH SIGNAL

DT/R':DATA TRANSMITT / DATA RECEIVED SIGNAL

- ✓ DT=1 IS HIGH-DATA IS TRASMITTED OUT
- ✓ R'=1 IS HIGH DATA IS RECEIVED IN

M/IO':MEMORY/INPUT OUTPUT

M=1 IS HIGH MEMORY IS ACCESSED

IO'=1 IS HIGH INPUT OUTPUTS ARE ACCESSED

WR':WRITE SIGNAL,IT IS A INPUT SIGNAL

IT IS A ACTIVE LOW SIGNAL AND WRITE DATA IN MEMORY

HLDA:HOLD ACKNOWLEDGE SIGNAL

IT IS ISSUED AFTER RECEIVING THE HOLD SIGNAL

HOLD:WHEN MICROPROCESSOR RECIEVES HOLD SIGNAL,IT ISSUES HLDA SIGNAL TO DMA(DIRECT MEMORY ACCESS) CONTROLLER.

MAXIMUM MODE(MX'):

- **IT IS ACTIVE LOW SIGNAL**

QS0,QS1:STATUS OF INSTRUCTION QUEUE

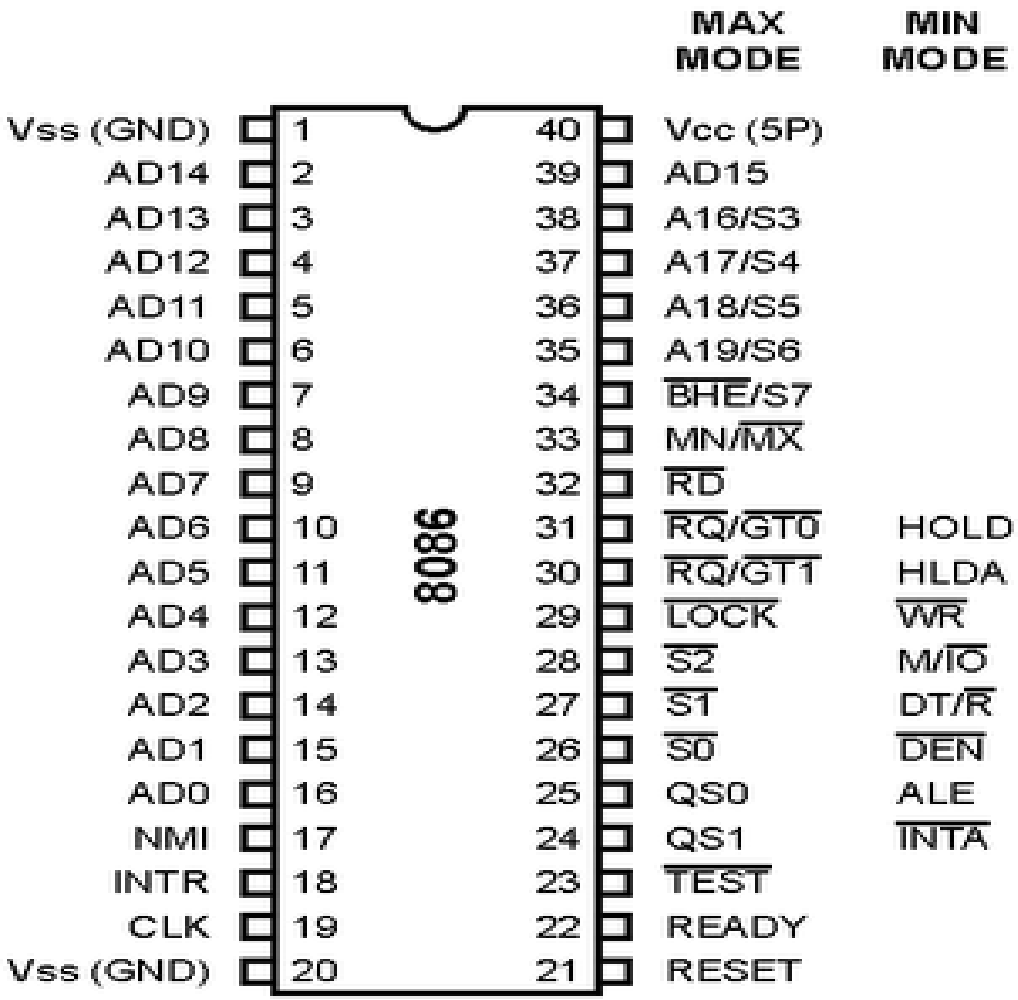
S0',S1',S2':STATUS SIGNALS INDICATE THE OPERATION BEING DONE BY THE MICROPROCESSOR.

LOCK'=LOCK PIN,IT IS ACTIVE LOW SIGNAL

RQ'/GT0' & RQ'/GT1':REQUEST /GRAND PINS

OTHER PROCESSORS REQUEST CPU AND AFTER RECEIVING REQUEST CPU SEND ACKNOWLEDGE SIGNAL.

RD':READ SIGNAL-OUTPUT SIGNAL


MAX MODE
MIN MODE
8086
Vss (GND) 1
AD14 2
AD13 3
AD12 4
AD11 5
AD10 6
AD9 7
AD8 8
AD7 9
AD6 10
AD5 11
AD4 12
AD3 13
AD2 14
AD1 15
AD0 16
NMI 17
INTR 18
CLK 19
Vss (GND) 20
40 Vcc (5P)
39 AD15
38 A16/S3
37 A17/S4
36 A18/S5
35 A19/S6
34 BHE/S7
33 MN/MX
32 RD
31 RQ/GT0 HOLD
30 RQ/GT1 HLDA
29 LOCK WR
28 S2 M/IO
27 S1 DT/R
26 S0 DEN
25 QS0 ALE
24 QS1 INTA
23 TEST
22 READY
21 RESET

MINIMUM AND MAXIMUM MODE OPERATIONS

8086 has two operating Modes:

I. Minimum mode

II. Maximum mode

Minimum mode:

- ✓ In this 8086 is the only processor in the system . In a minimum mode 8086 system.
- ✓ 8086 is operated in minimum mode when MN/MX' pin to logic 1.
- ✓ In this mode, all the control signals are given out by the 8086 itself.

Maximum mode:

- ✓ In this we can connect more processors to 8086 (8087/8089).
- ✓ 8086 max mode is basically for implementation of allocation of global resources and passing bus control to other coprocessor(i.e. second processor in the system), because two processors can not access system bus at same instant.
- ✓ All processors execute their own program.
- ✓ The resources which are common to all processors are known as global resources.
- ✓ The resources which are allocated to a particular processor are known as local or private resources.

MINIMUM MODE OPERATION OF 8086

- ✓ The 8086 microprocessor operates in minimum mode when MN/MX' = 1.
- ✓ In minimum mode,8086 is the only processor in the system which provides all the control signals which are needed for memory operations and I/O interfacing.
- ✓ Here the circuit is simple but it does not support multiprocessing.
- ✓ The other components which are transceivers, latches, 8284 clock generator, 74138 decoder, memory and i/o devices are also present in the system.
- ✓ The address bus of 8086 is 20 bits long. By this we can access 2^20 byte memory i.e. 1MB . Out of 20 bits, 16 bits A0 to A15(or 16 lines) are multiplexed with a data bus. By multiplexing, it means they will act as address lines during the first T state of the machine cycle and in the rest, they act as data lines. A16 to A19 are multiplexed S3 to S6 and BHE' is multiplexed with S7.

Control signals provided by 8086 for memory operations and i/o interfacing :

- ✓ They are used to identifying whether the bus is carrying a valid address or not , in which direction data is needed to be transferred over the bus, when there is valid write data on the data bus and when to put read data on the system bus.
- ✓ Therefore, their sequence pattern makes all the operations successful in a particular machine cycle.

8282 (8 bits) latch :

The latches are buffered D FF. They are used to separate the valid address from the multiplexed Address/data bus by using the control signal ALE, which is connected to strobe(STB) of 8282. The ALE is active high signal. Here three such latches are required because the addresis 20 bit

8286 (8 bits) transceivers :

They are bidirectional buffers and also known as data amplifiers. They are used to separate the valid data from multiplexed add/data bus. Two such transceivers are needed because the data bus is 16 bits long. 8286 is connected to DT/R' and DEN' signals. They are enabled through the DEN signal .The direction of data on the data bus is controlled by the DT/R' signal. DT/R' is connected to T and DEN' is connected to OE'.

DEN'	DT/R'	Action
1	X (don't care)	Transreceiver is disabled
0	0	Receiver data
0	1	Transmit data

- ✓ 8284 clock generator is used to provide the clock.
- ✓ M/IO'= 1,then I/O transfer is performed over the bus. and when M/IO' = 0, then I/O operation is performed.
- ✓ The signals RD' and write WR' are used to identify whether a read bus cycle or a write bus cycle is performing. When WR' = 0 ,then it indicates that valid output data on the data bus.
- ✓ RD' indicates that the 8086 is performing a read data or instruction fetch process is occurring .During read operations, one other control signal is also used, which is DEN (data enable) and it indicates the external devices when they should put data on the bus.
- ✓ Control signals for all operations are generated by decoding M/IO', RD', WR'. They are decoded by 74138 3:8 decoder.

M/IO'	RD'	WR'	Action
1	0	1	Memory Read
1	1	0	Memory Write
0	0	1	I/O Read
0	1	0	I/O Write

INTR and INTA :

When INTR = 1,then there is an interrupt to 8086 by other devices for their service.

When INTA'= 0,then it indicates that the processor is ready to service them.

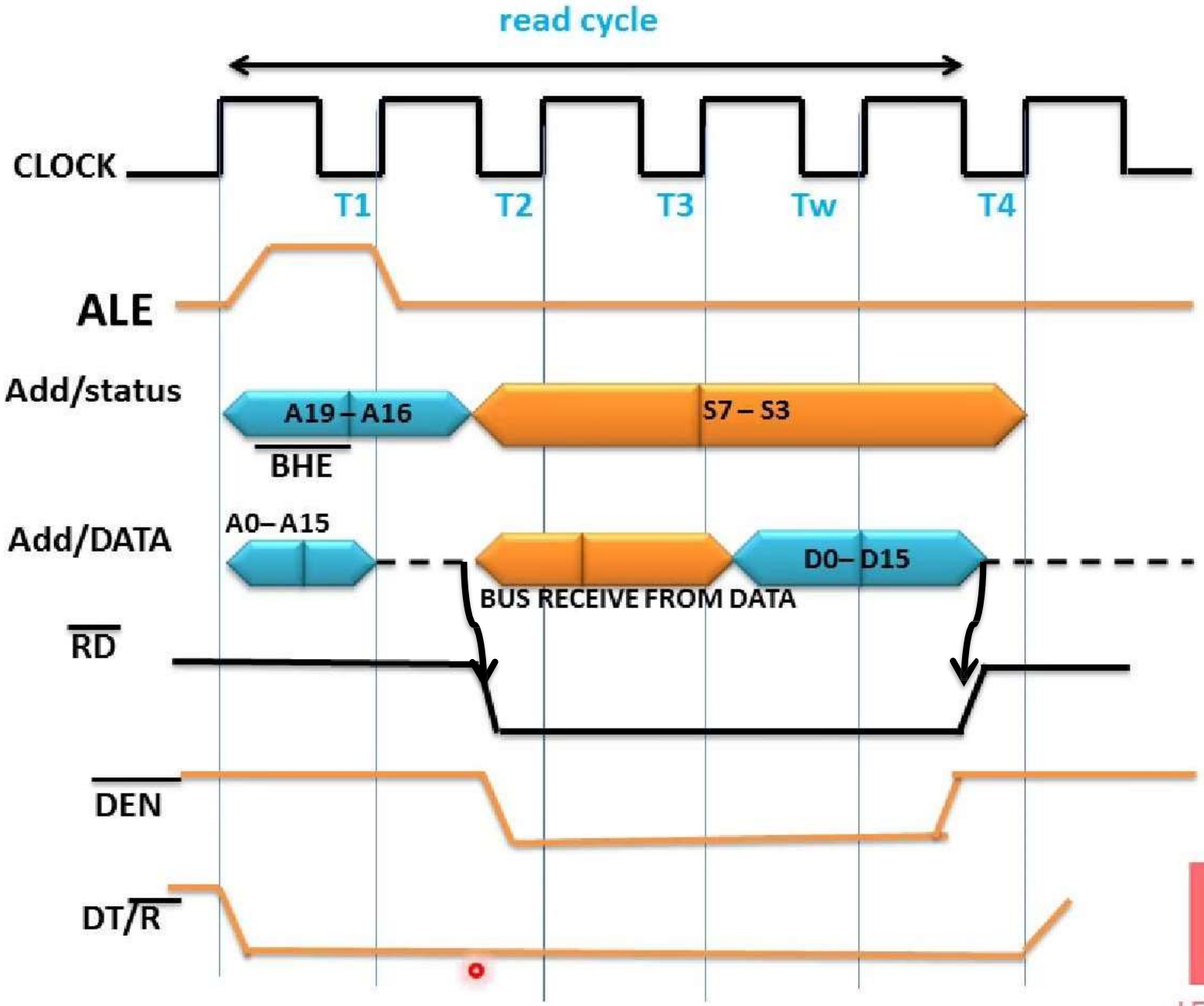
The bus request is made by other devices using the HOLD signal and the processor acknowledges them using the HLDA output signal.

Read Cycle

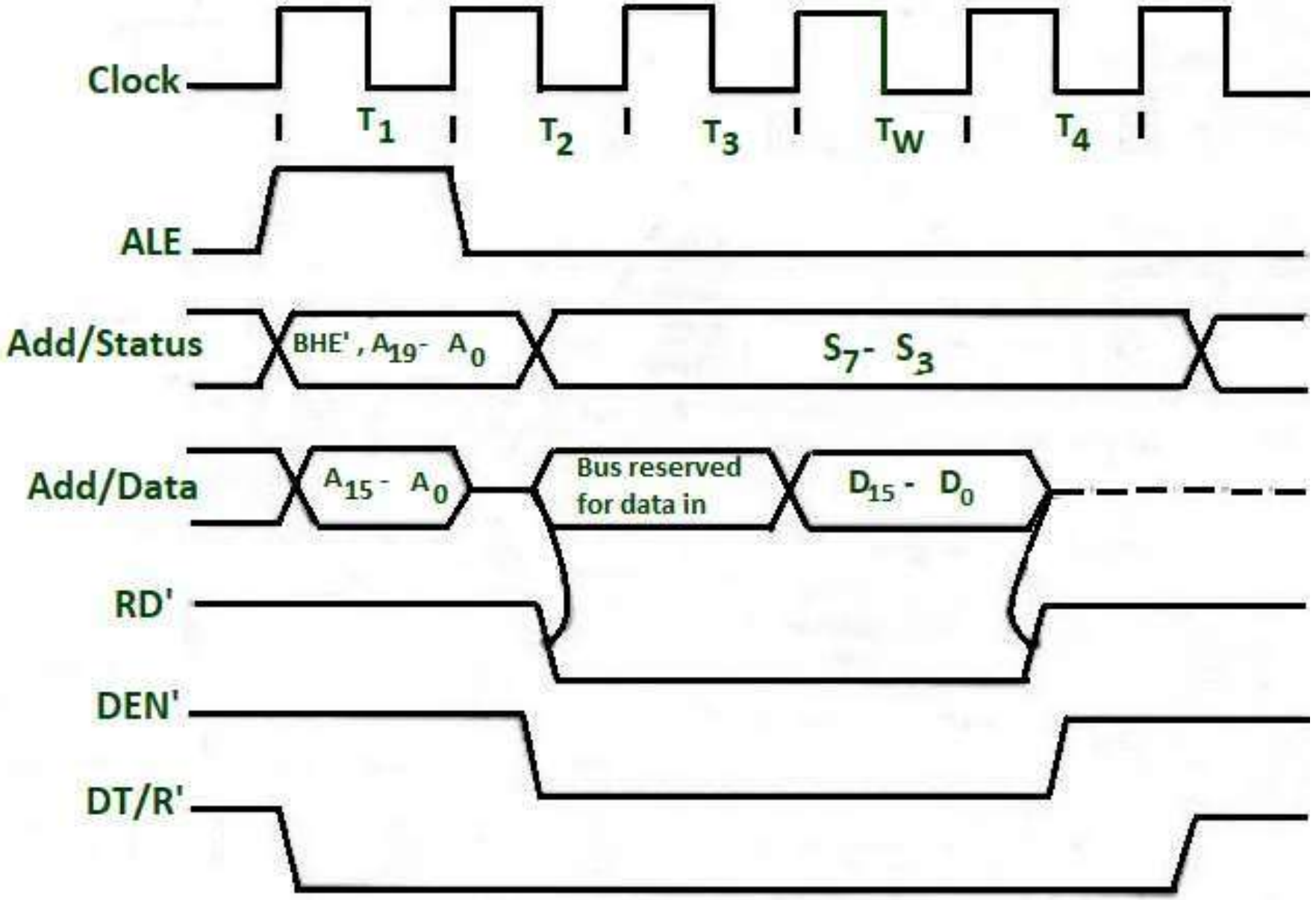
Timing diagram :

- ✓ The working of min mode can be easily understood by timing diagrams.
- ✓ All processors bus cycle is of at least 4 T-states(T_1,T_2,T_3,T_4) .The address is given by processor in the T1 state. It is available on the bus for **one T-state.**
- ✓ In T_2, the bus is tristated for changing the direction of the bus(in the case of a data read cycle.)
- ✓ The data transfer takes place between T_3 and T_4.
- ✓ If the addressed device is slower, then the wait state is inserted between T_3 and T_4.

READ CYCLE TIMING DIAGRAM MINIMUM MODE 8086 SYSTEM



LAG
LEARN AND GROW

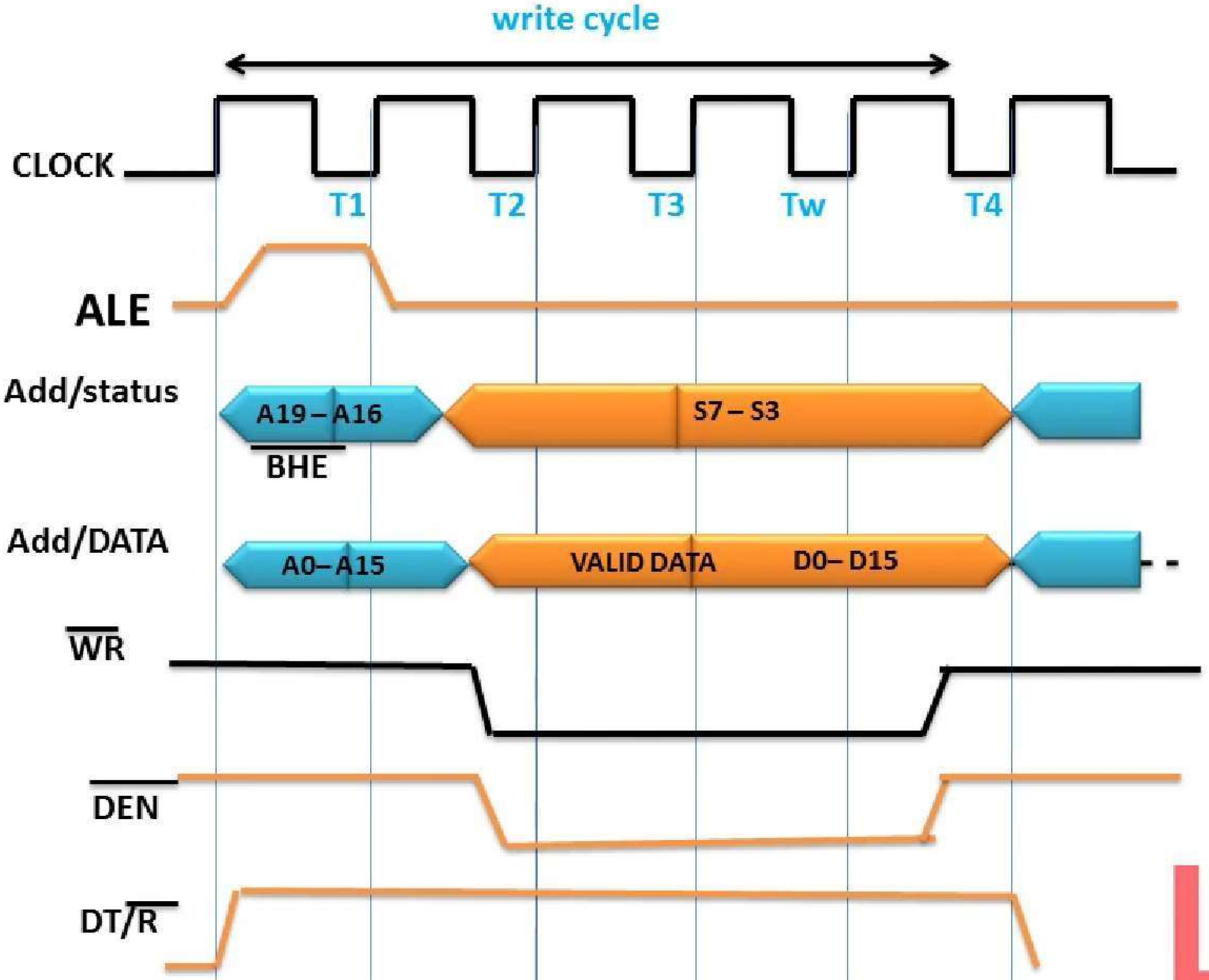

Clock
T1
T2
T3
TW
T4
ALE
Add/Status
BHE' , A19- A0
S7- S3
Add/Data
A15 - A0
Bus reserved
for data in
D15 - D0
RD'
DEN'
DT/R'

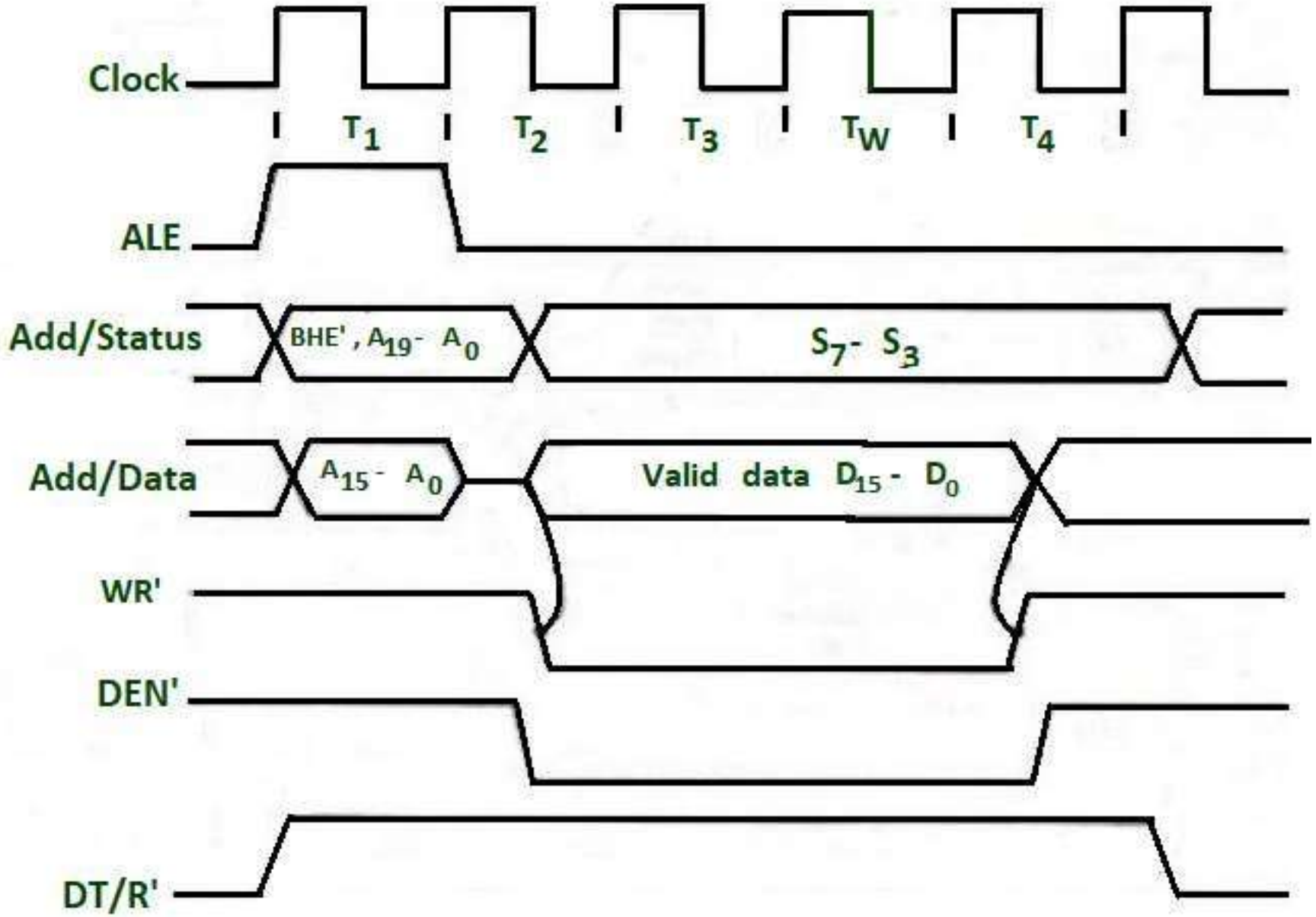
- ✓ At T1 state ALE =1 ,this indicates that a valid address is latched on the address bus and also M / IO'= 1, which indicates the memory operation is in progress.
- ✓ In T2, the address is removed from the local bus and is sent to the addressed device. Then the bus is tristated.
- ✓ When RD' = 0 , the valid data is present on the data bus.
- ✓ During T2 DEN' =0, which enables transceivers and DT/R' = 0 ,which indicates that the data is received.
- ✓ During T3, data is put on the data bus and the processor reads it.
- ✓ The output device makes the READY line high. This means the output device has performed the data transfer process. When the processor makes the read signal to 1, then the output device will again tristate its bus drivers.

Write Cycle

- ✓ At T1 state ALE =1 ,this indicates that a valid address is latched on the address bus and also M / IO'= 1, which indicates the memory operation is in progress.
- ✓ In T2, the processor sends the data to be written to the addressed location.
- ✓ The data is buffered on the bus until the middle of T4 state.
- ✓ The WR'=0 becomes at the beginning of T2.
- ✓ The BHE' and A0 signals are used to select the byte or bytes of memory or I/O word.
- ✓ During T2 DEN' =0, which enables, transceivers and DT/R' = 1 ,which indicates that the data is transferred by the processor to the addressed device.

WRITE CYCLE TIMING DIAGRAM MINIMUM MODE 8086 SYSTEM




Clock
T1
T2
T3
TW
T4
ALE
Add/Status
BHE', A19- A0
S7- S3
Add/Data
A15 - A0
Valid data D15 - D0
WR'
DEN'
DT/R'

MAXIMUM MODE 8086 SYSTEM

- ✓ When MN/ MX' = 0 , 8086 works in max mode.
- ✓ Clock is provided by 8284 clock generator.
- ✓ 8288 bus controller- Address form the address bus is latched into 8282 8-bit latch. Three such latches are required because address bus is 20 bit.
- ✓ The ALE(Address latch enable) is connected to STB(Strobe) of the latch. The ALE for latch is given by 8288 bus controller.
- ✓ The data bus is operated through 8286 8-bit transceiver.
- ✓ Two such transceivers are required, because data bus is 16-bit. The transceivers are enabled the DEN signal, while the direction of data is controlled by the DT/R signal.
- ✓ DEN is connected to OE' and DT/ R' is connected to T. Both DEN and DT/ R' are given by 8288 bus controller.

DEN (Of 8288)	DT/ R'	Action
0	X	Transreceiver is disabled
1	0	Receive data
1	1	Transmit data

- ✓ Bus request is done using RQ' / GT' lines interfaced with 8086. RQ0/GT0 has more priority than RQ1/GT1.
- ✓ INTA' is given by 8288, in response to an interrupt on INTR line of 8086.
- ✓ In max mode, the advanced write signals get enabled one T-state in advance as compared to normal write signals. This gives slower devices more time to get ready to accept the data, therefore it reduces the number of cycles.

S'_2	S'_1	S'_0	**Processor State (What the µP wants to do)**	**8288 Active Output (What Control signal should 8288 generate)**
0	0	0	Interrupt Acknowledge	**INTA'**
0	0	1	Read I/O Port	**IORC'**
0	1	0	Write I/O Port	**IOWC' and AIOWC'**
0	1	1	Halt	None
1	0	0	Instruction Fetch	**MRDC'**
1	0	1	Memory Read	**MRDC'**
1	1	0	Memory Write	**MWTC' and AMWTC'**
1	1	1	Inactive	None

d) QS0 and QS1:

- These two are output pins are used to indicate the instruction queue status.

QS1	QS0	Queue Operation
0	0	NOP
0	1	Opcode Fetch from queue
1	0	Queue is Cleared
1	1	Fetch remaining instruction bytes form queue

MRDC (Memory Read Command) : It instructs the memory to put the contents of the addressed location on the data bus.

MWTC (Memory Write Command) : It instructs the memory to accept the data on the data bus and load the data into the addressed memory location.

IORC (I/O Read Command) : It instructs an I/O device to put the data contained in the addressed port on the data bus.

IOWC (I/0 Write Command) : It instructs an I/O device to accept the data on the data bus and load the data into the addressed port.

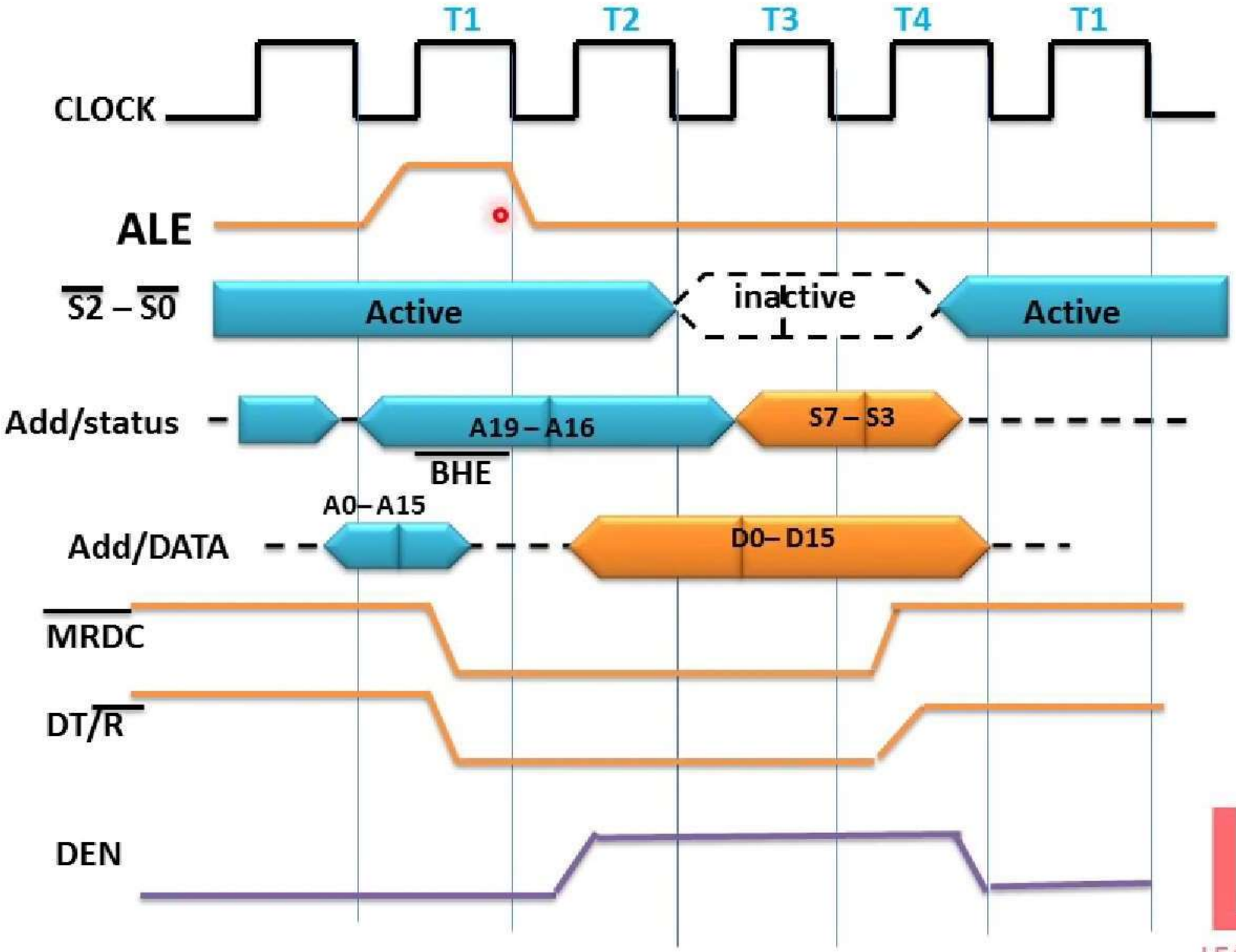
MCE/PDEN (Master Cascade Enable/Peripheral Data Enable) : It controls the mode of operation of 8259. It selects cascade operation for 8259 (interrupt controller) if IOB signal is grounded and enables the I/O bus transceivers if IOB is tied high.

AEN, IOB and CEN : These pins are used in multiprocessor system. With a single processor in the system, AEN and IOB are grounded and CEN is tied high. AEN causes the 8288 to enable the memory control signals. IOB (I/O bus mode) signal selects either the I/O bus mode or system bus mode operation. CEN (control enable) input enables the command output pins on the 8288.

AIOWC/AMWC (Advance I/O Write Command/Advance Memory Write Command) : These signals are similar to IOWC and MWTC except that they are activated one clock pulse earlier. This gives slow interfaces an extra clock cycle to prepare to input the data.

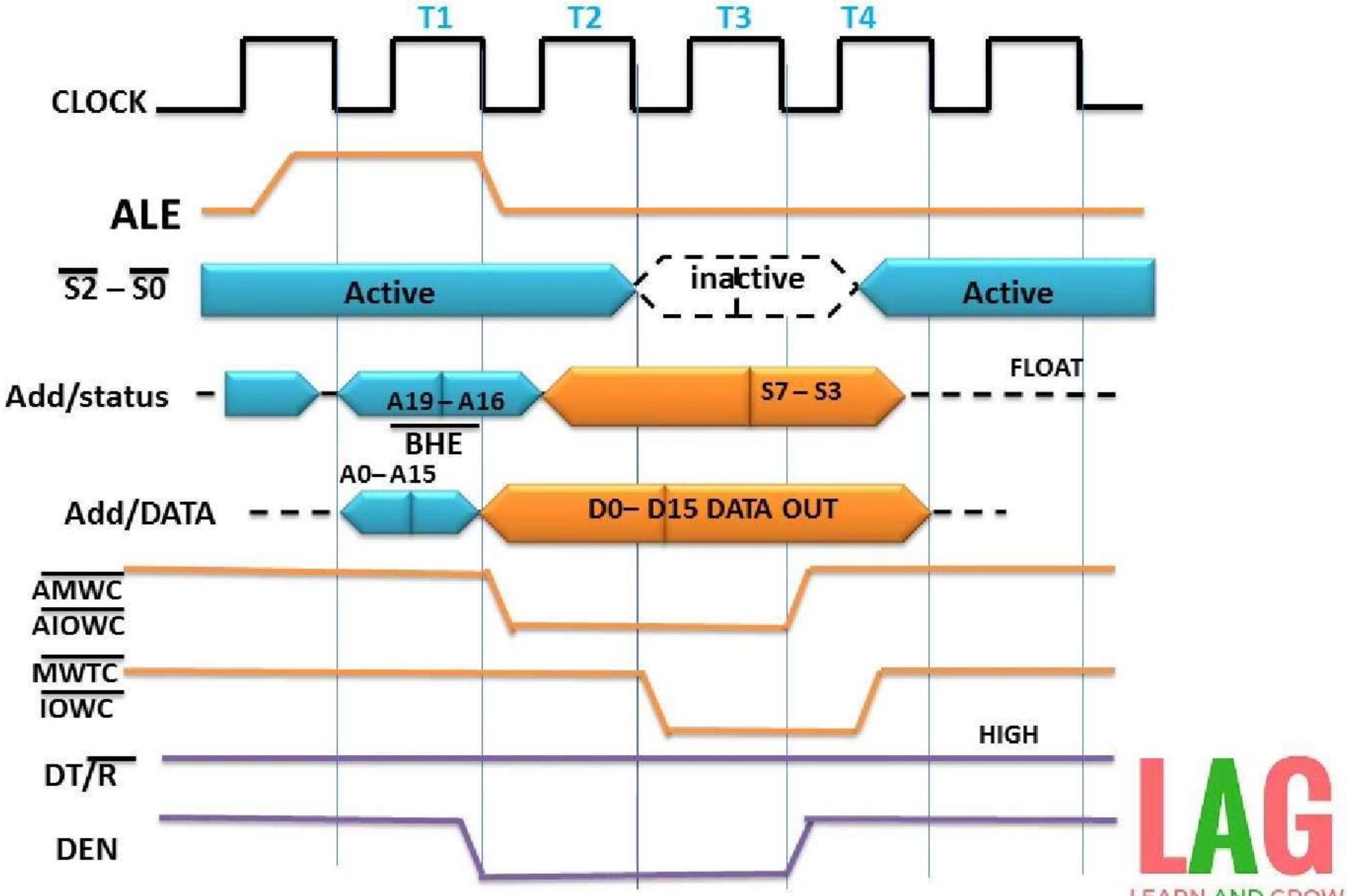
- ✓ S0,S1,S2 are set at the beginning of bus cycle. On detecting the change on
- ✓ passive state S0 = S1 = S2 = 1, the 8288 bus controller will output a pulse on its ALE and apply a required signal to its DT/R pin during T1.
- ✓ In T2, 8288 will set DEN = 1 thus enabling transceiver. For an input, 8288 it will activates MRDC or IORC. These signals are activated until T4. For an output, the AMWC or AIOWC is activated from T2 to T4 and MWTC or IOWC is activated from T3 to T4.
- ✓ The status bits S0 to S2 remain active until T3, and become passive during T3 and T4.
- ✓ If ready input is not activated before T3, wait state will be inserted between T3 and T4.

MEMORY READ CYCLE TIMING DIAGRAM MAXIMUM MODE 8086 SYSTEM



LAG
LEARN AND GROW

MEMORY WRITE CYCLE TIMING DIAGRAM MAXIMUM MODE 8086 SYSTEM



LAG
LEARN AND GROW

ADDRESSING MODES OF 8086

- The method of specifying data to be operated by an instruction is called as addressing mode.
- The different ways that a processor can access the data are referred to as addressing modes.

- Every instruction of a program has to operate on a data.
- The different ways in which a source operand is denoted in an instruction are known as addressing modes.

Addressing Modes

1. Register Addressing 2. Immediate Addressing	Group I : Addressing modes for register and immediate data
3. Direct Addressing 4. Register Indirect Addressing 5. Based Addressing 6. Indexed Addressing 7. Based Index Addressing 8. String Addressing	Group II : Addressing modes for memory data
9. Direct I/O port Addressing 10. Indirect I/O port Addressing	Group III : Addressing modes for I/O ports
11. Relative Addressing	Group IV : Relative Addressing mode
12. Implied Addressing	Group V : Implied Addressing mode

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

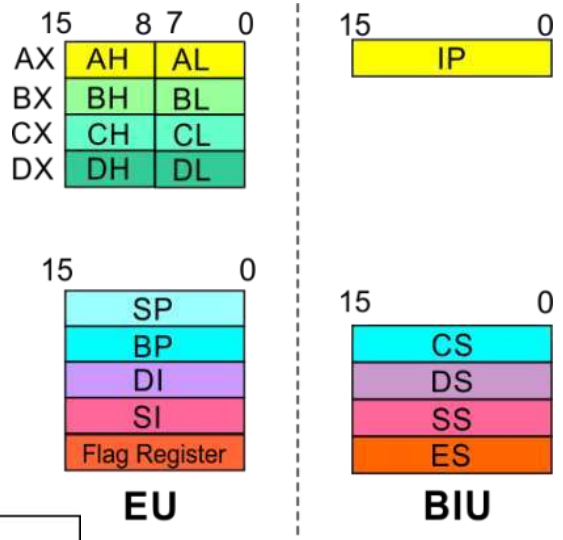
The instruction will specify the name of the register which holds the data to be operated by the instruction.

Example:

MOV CL, DH

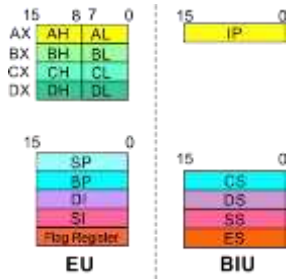
The content of 8-bit register DH is moved to another 8-bit register CL

(CL) ← (DH)





1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In immediate addressing mode, an 8-bit or 16-bit data is specified as part of the instruction

Example:

MOV DL, 08H

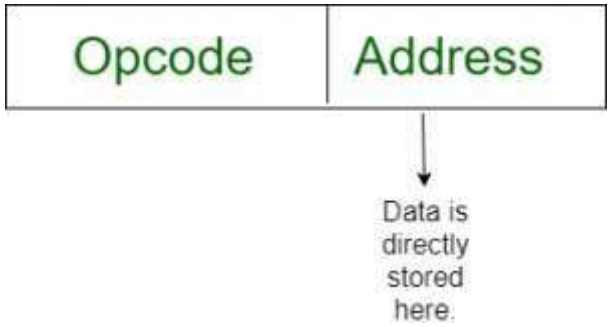
The 8-bit data (08_H) given in the instruction is moved to DL

(DL) ← 08_H

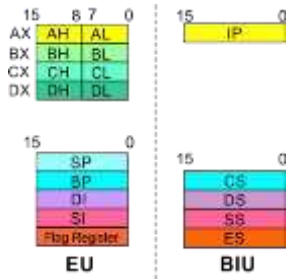
MOV AX, 0A9FH

The 16-bit data ($0A9F_H$) given in the instruction is moved to AX register

(AX) ← $0A9F_H$



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



Here, the effective address of the memory location at which the data operand is stored is given in the instruction.

The effective address is just a 16-bit number written directly in the instruction.

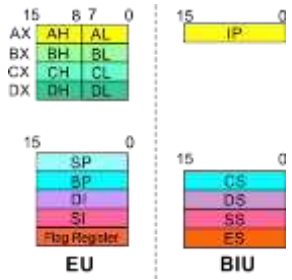
Example:

```
MOV   BX, [1354H]
MOV   BL, [0400H]
```

The square brackets around the 1354_H denotes the contents of the memory location. When executed, this instruction will copy the contents of the memory location into BX register.

This addressing mode is called direct because the displacement of the operand from the segment base is specified directly in the instruction.

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In Register indirect addressing, name of the register which holds the effective address (EA) will be specified in the instruction.

Registers used to hold EA are any of the following registers:

BX, BP, DI and SI.

Content of the DS register is used for base address calculation.

Example:

MOV CX, [BX]

Note : Register/ memory enclosed in brackets refer to content of register/ memory

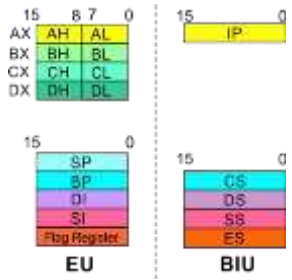
Operations:

EA = (BX)
BA = (DS) x 16_{10}
MA = BA + EA

(CX) ← (MA) or,

(CL) ← (MA)
(CH) ← (MA +1)

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



✓ Based Addressing Mode

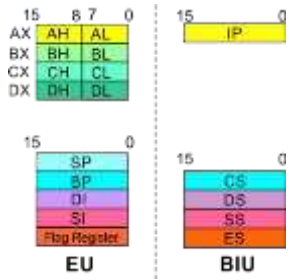
✓ Based Addressing specifies a signed 8-bit or an unsigned 16-bit displacement and stores the base value for the effective address in BX or BP.

✓ When there is an 8-bit displacement, the sign is 16-bit extended before being added to the base value.

MOV AX, [BX + 04]

Given instruction indicates that there is a value stored in the BX register and this is added with 4 bits displacement value and will store in the AX register.

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



Indexed Mode

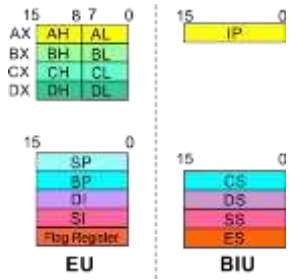
The SI or DI register stores the index value for memory data, and the instruction will specify either a signed 8-bit or an unsigned 16-bit displacement.

To produce the EA, displacement is added to the index value in the SI or DI register.

MOV CX, [SI + 16]

Given instruction indicates that there is a value stored in the SI (Source Index) register and this is added with 16 bits displacement value and will store in the BX register.

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



Based Indexed Mode

In the based index addressing of 8086, a base register (BX or BP), an index register (SI or DI), and a displacement are added to determine the effective address.

Examples of Based Indexed Addressing Modes are given below.

MOV DX, [BX + SI]

Given instruction indicates that there is a value stored in the SI (Source Index) register and this is added to the value of the BX register and will store in the DX register.

Based Indexed Displacement Mode

MOV AX, [BX+SI+08]

Given instruction indicates that there is a value stored in the SI (Source Index) register and this is added to the value of the BX register along with the 8-bit displacement value and will store in the AX register.

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

Note : Effective address of the Extra segment register

This addressing mode is related to string instructions. In this the value of SI and DI are auto incremented and decremented depending upon the value of directional flag.

Example:

MOVS B

MOVS W

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

IN AL, [80H]

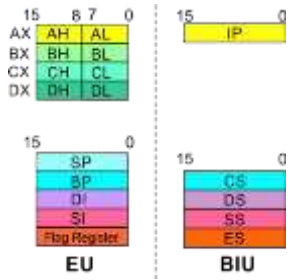
It reads one byte of data from I/O port address

80H to AL

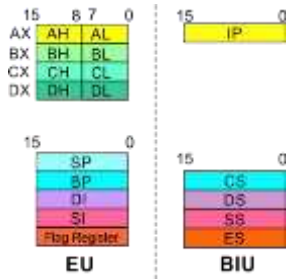
OUT [82H], AL

It sends one byte of data from AL to I/O port

address 82H



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In this addressing mode, the effective address of a program instruction is specified relative to Instruction Pointer (IP) by an 8-bit signed displacement.

Example: JZ 0AH

Jump if zero

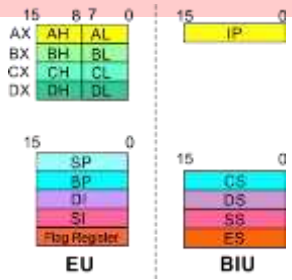
JNZ Jum if not zero

JNC Jum if not carry

If ZF = 1, then the program control jumps to new address calculated above.

If ZF = 0, then next instruction of the program is executed.

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In this Addressing mode, the operand is implied in instruction

Example:

CLC

This clears the carry flag to zero.

STC

Set the carry flag

CLD

Clear directional flag

Find Physical Address ???

- ✓ **Determine the physical address when CS=5300H and IP=0200H**
- ✓ **Determine the physical address when ES=6500H and IP=4767H**
- ✓ **Determine the physical address when CS=24F6H and IP=634AH**
- ✓ **DS locate physical address of 43657H. Assume IP is 2057H**
- ✓ **What is the componenet of data segment DS to locate the Physical address of 43657H.Assume the content of IP is 2057H**

8086 MICROPROCESSOR MODES OF OPERATION

8086 has two operating Modes:

I. Minimum mode

II. Maximum mode

Minimum mode:

- ✓ In this 8086 is the only processor in the system . In a minimum mode 8086 system.
- ✓ 8086 is operated in minimum mode when MN/MX' pin to logic 1.
- ✓ In this mode, all the control signals are given out by the 8086 itself.

Maximum mode:

- ✓ In this we can connect more processors to 8086 (8087/8089).
- ✓ 8086 max mode is basically for implementation of allocation of global resources and passing bus control to other coprocessor(i.e. second processor in the system), because two processors can not access system bus at same instant.
- ✓ All processors execute their own program.
- ✓ The resources which are common to all processors are known as global resources.
- ✓ The resources which are allocated to a particular processor are known as local or private resources.

INSTRUCTIONS OF 8086

OR

INSTRUCTION SETS OF 8086

- An instruction is a binary pattern designed inside a microprocessor to perform a specific function.
- The entire group of instructions that a microprocessor supports is called **Instruction Set.**
- 8086 has more than **20,000** instructions.

The 8086 microprocessor supports 8 types of instructions –

1. Arithmetic Instructions
2. Data Transfer Instructions
3. Logical Instructions
4. Control Transfer Instructions
5. String Manipulation Instructions
6. Processor Control Instructions

1.Arithmetic instructions in 8086 microprocessor

- ✓ Arithmetic Instructions are the instructions which perform basic arithmetic operations such as addition, subtraction and a few more.
- ✓ Following is the table showing the list of arithmetic instructions:

Program 9: Add two 8-bit numbers

MOV AL, 05H	Move 1st 8-bit number to AL.
MOV BL, 03H	Move 2nd 8-bit number to BL.
ADD AL, BL	Add BL with AL.

Program 10: Add two 16-bit numbers

MOV AX, 0005H	Move 1st 16-bit number to AX.
MOV BX, 0003H	Move 2nd 16-bit number to BX.
ADD AX, BX	Add BX with AX.

Program 11: subtract two 8-bit numbers

MOV AL, 05H	Move 1st 8-bit number to AL.
MOV BL, 03H	Move 2nd 8-bit number to BL.
SUB AL, BL	**subtract** BL from AL.

Program 12: subtract two 16-bit numbers

MOV AX, 0005H	Move 1st 16-bit number to AX.
MOV BX, 0003H	Move 2nd 16-bit number to BX.
SUB AX, BX	**subtract** BX from AX.

Program 15: Multiply two 16-bit unsigned numbers.

MOV AX, 0004H	Move 1st 16-bit number to AL.
MOV BX, 0002H	Move 2nd 16-bit number to BL.
MUL BX	Multiply BX with AX and the result will

be in **DX:AX** **{4*2=0008=> 08=> AX , 00=> DX}**

Program 16: Divide two 16-bit unsigned numbers.

MOV AX, 0004H	Move 1st 16-bit number to AL.
MOV BX, 0002H	Move 2nd 16-bit number to BL.
DIV BX	Divide BX from AX and the result will be in **AX & DX**

{4/2=0002=> 02=> AX ,00=>DX}
(ie: Quotient => AX , Reminder => DX)

Program 13: Multiply two 8-bit unsigned numbers.

MOV AL, 04H	Move 1st 8-bit number to AL.
MOV BL, 02H	Move 2nd 8-bit number to BL.
MUL BL	Multiply BL with AL and the result will be in AX.

Program 14: Multiply two 8-bit signed numbers.

MOV AL, 04H	Move 1st 8-bit number to AL.
MOV BL, 02H	Move 2nd 8-bit number to BL.
IMUL BL	Multiply BL with AL and the result will be in AX.

Program 1: Increment an 8-bit number

- MOV AL, 05H — Move 8-bit data to AL.
- INC AL — Increment AL.

Program 2: Increment an 16-bit number

- MOV AX, 0005H — Move 16-bit data to AX.
- INC AX — Increment AX.

Program 3: Decrement an 8-bit number

- MOV AL, 05H — Move 8-bit data to AL.
- DEC AL — Decrement AL.

Program 4: Decrement an 16-bit number

- MOV AX, 0005H — Move 16-bit data to AX.
- DEC AX — Decrement AX.

2. Data Copy/ Transfer Instructions

I. MOV: Transfer data from 1 register/ memory location to other register/memory location.

Eg. MOV AX,[2000H]. . . . Direct

MOV AX,BX . . . . .Register

II IN(Input the Port): User for reading an input port

AL & AX are allowed destinations for 8 & 16 bit input instructions.

Ex. IN AL,03h . . . Reads data from 8 bit port whose address is 03H & store in AL.

III OUT(Output to the Port): User for writing to an output port

Ex. OUT 03H, AL . . . Sends data available in AL to port whose address is 03H .

VI. XCHG(Exchange) – XCHG Destination, Source

1. Exchanges the content of a register with the content of another register or with the content of memory location(s).
2. It cannot directly exchange the content of two memory locations.
3. The source and destination must both be of the same type (bytes or words).
4. The segment registers cannot be used in this instruction.
5. This instruction does not affect any flag.

Ex. XCHG AX, DX . . . . Exchange word in AX with word in DX

3.Logical Instructions

LOGIC instruction =

These instruction use to perform logic operations AND , OR , NOT etc.

1 = AND : LOGIC AND

AND instruction source operand immediate , register or memory location to the destination operand

```
AND     AX , 0008H   content of AX if 3F0FH
ADD     AX  , BX
```

```
0 0 1 1   1 1 1 1   0 0 0 0   1 1 1 1     = 3F0F H  [AX]
                                            AND
0 0 0 0   0 0 0 0   0 0 0 0   1 0 0 0     = 0008 H
-------------------------------------------------------
0 0 0 0   0 0 0 0   0 0 0 0   1 0 0 0     = 0008 H  [AX]
```

2 = OR : LOGIC OR

OR instruction source operand immediate , register or memory location to the destination operand

OR AX , 0098H content of AX if 3F0FH

OR AX , BX

0 0 1 1 1 1 1 1 0 0 0 0 1 1 1 1 = 3F0F H [AX]

OR

0 0 0 0 0 0 0 0 1 0 0 1 1 0 0 0 = 0098 H

---

0 0 1 1 1 1 1 1 1 0 0 1 1 1 1 1 = 3F9F H [AX]

3 = NOT : LOGIC NOT

NOT instruction show the complement of operant register or memory location

NOT AX content of AX if 3F0FH

NOT [5000H]

0 0 1 1 1 1 1 1 0 0 0 0 1 1 1 1 = 3F0F H [AX]

NOT

1 1 0 0 0 0 0 0 1 1 1 1 0 0 0 0

C 0 F 0

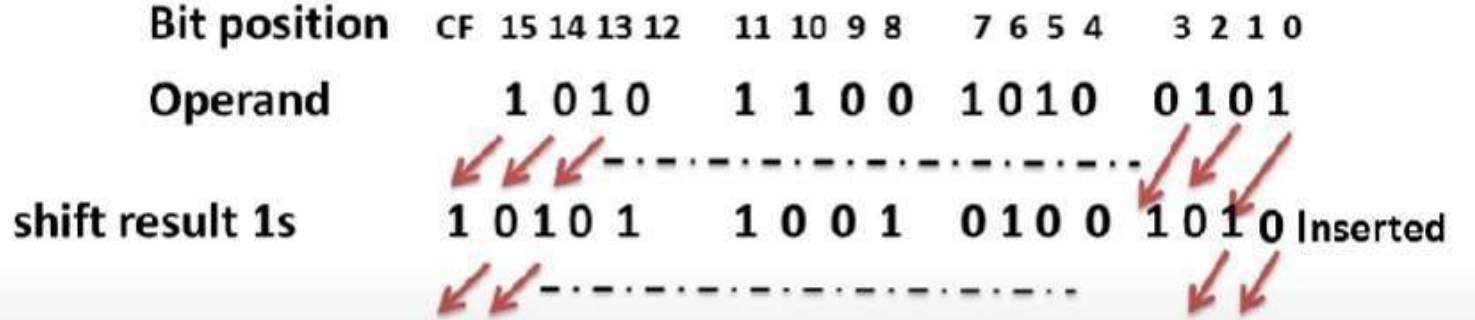
4 = XOR : LOGIC EXCLUSIVE OR

```
XOR     AX , 0098H   content of AX if 3F0FH
XOR     AX , BX
```

```
0 0 1 1   1 1 1 1   0 0 0 0   1 1 1 1     = 3F0F H  [AX]
                                            XOR
0 0 0 0   0 0 0 0   1 0 0 1   1 0 0 0     = 0098 H
-------------------------------------------------------
0 0 1 1   1 1 1 1   1 0 0 1   0 1 1 1     = 3F97 H  [AX]
```

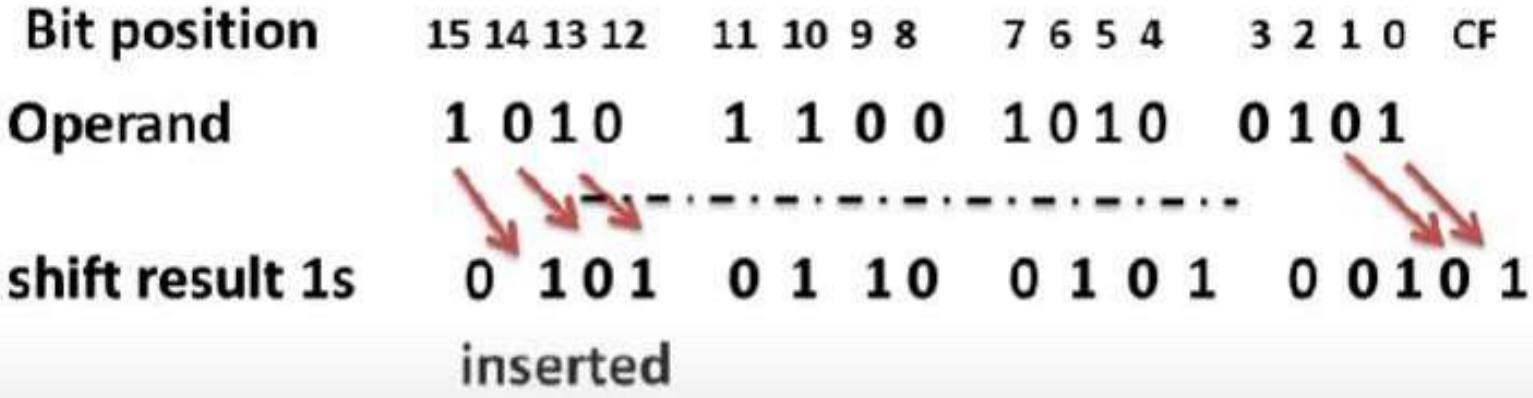
5 = SHL/SAL : SHIFT LOGIC /ARITHMETIC LEFT

SHL/SAL shift operand word left bit by bit and insert 0 in LSB



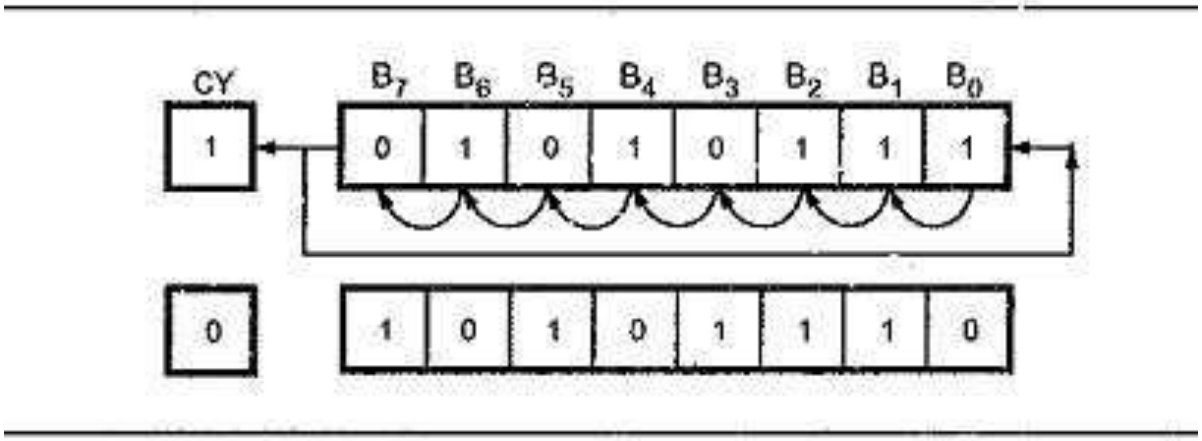
6 = SHR : SHIFT LOGIC RIGHT

SHR shift operand word right



7.ROL Des, Count:

- It rotates bits of byte or word left, by count.
- MSB is transferred to LSB and also to CF.

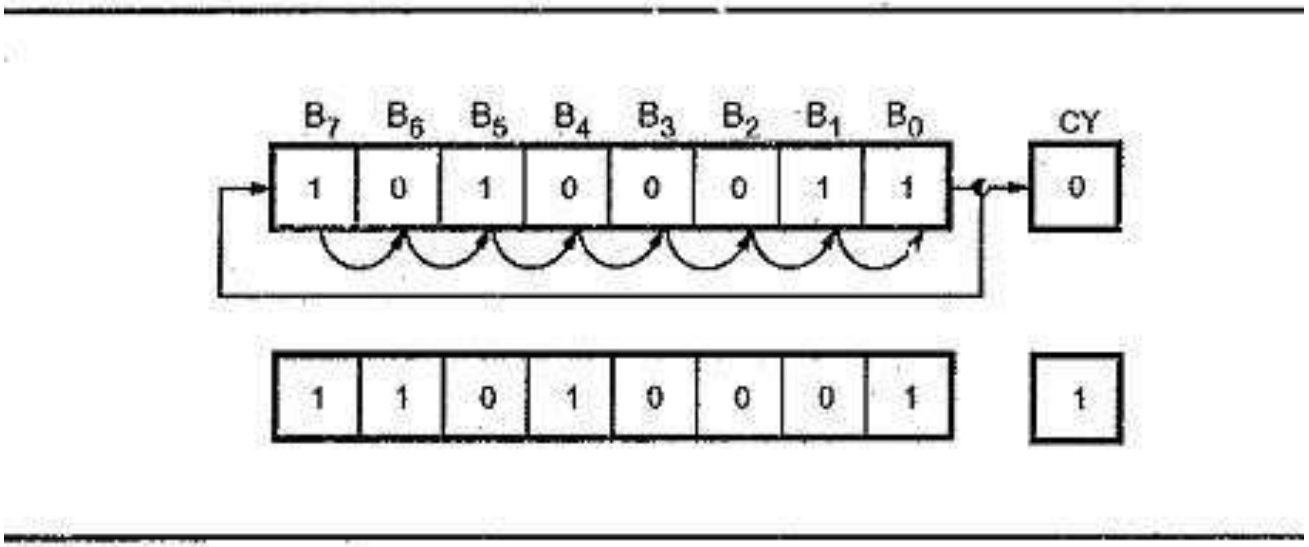


Examples :

```
ROL  CX,  1        ; Word in CX one bit position left, MSB to
                   ; LSB and CF
MOV  CL,  03H      ; Load desired number of bits to rotate in CL.
ROL  BL,  CL       ; Rotate BL three positions.
```

8.ROR Des, Count:

- It rotates bits of byte or word right, by count.
- LSB is transferred to MSB and also to CF.



Examples :

```
ROR  CX,  1          ;  Rotated word in CX one bit position
                     ;  left, LSB to MSB and CF.
MOV  CL,  03H        ;  Load number of bits to rotate in CL.
ROR  BL,  CL         ;  Rotate BL three positions.
```

4.Control Transfer Instructions

- These instructions cause change in the sequence of the execution of instruction.
- This change can be through a condition or sometimes unconditional.
- The conditions are represented by flags.

- **CALL Des:**
  - This instruction is used to call a subroutine or function or procedure.
  - The address of next instruction after CALL is saved onto stack.
- **RET:**
  - It returns the control from procedure to calling program.
  - Every CALL instruction should have a RET.
- **JMP Des:**
  - This instruction is used for unconditional jump from one place to another.
- **Jxx Des (Conditional Jump):**
  - All the conditional jumps follow some conditional statements or any instruction that affects the flag.

Conditional Jump Table

Mnemonic	Meaning	Jump Condition
JA	Jump if Above	CF = 0 and ZF = 0
JAE	Jump if Above or Equal	CF = 0
JB	Jump if Below	CF = 1
JBE	Jump if Belowor Equal	CF = 1 or ZF = 1
JC	Jump if Carry	CF = 1
JE	Jump if Equal	ZF = 1
JNC	Jump if Not Carry	CF = 0
JNE	Jump if Not Equal	ZF = 0
JNZ	Jump if Not Zero	ZF = 0
JPE	Jump if Parity Even	PF = 1
JPO	Jump if Parity Odd	PF = 0
JZ	Jump if Zero	ZF = 1

5.String Manipulation Instructions

- String in assembly language is just a sequentially stored bytes or words.
- There are very strong set of string instructions in 8086.
- By using these string instructions, the size of the program is considerably reduced.

- **CMPS Des, Src:**
  - It comparesthe string bytes or words.
- **SCAS String:**
  - It scans a string.
  - It comparesthe String with byte in AL or with word in AX.
- **MOVS / MOVSB / MOVSW:**
  - It causes moving of byte or word from onestring to another.
  - In this instruction, the source string is in Data Segment and destination string is in Extra Segment.
  - SI and DI store the offsetvalues for source and destination index.

6.Processor Control Instructions

- These instructions control the processor itself.
- 8086 allows to control certain control flags that:
  - causes the processing in a certain direction
  - processor synchronization if more than one microprocessor attached.

- **STC:**
  - It sets the carry flag to 1.
- **CLC:**
  - It clears the carry flag to 0.
- **CMC:**
  - It complements the carry flag.
- **STD:**
  - It sets the direction flag to 1.
  - If it is set, string bytes are accessed from higher memory address to lower memory address.
- **CLD:**
  - It clears the direction flag to 0.
  - If it is reset, the string bytes are accessed from lower memory address to higher memory address.

Assembler Directives

- **Data Definition Directives:**
  - DB (Define Byte)
  - DW (Define Word)
  - DD (Define Doubleword)
  - DQ (Define Quadword)
- **Segment Declaration Directives:**
  - SEGMENT
  - ENDS
- **Procedure Declaration Directives:**
  - PROC
  - ENDP
- **Assembly Control Directives:**
  - ASSUME
  - END

ASSEMBLER DIRECTIVES

- Assembler directives are the commands to the assembler that direct the assembly process.
- They indicate how an operand is treated by the assembler and how assembler handles the program.
- They also direct the assembler how program and data should arrange in the memory.
- ALP's are composed of two type of statements.

  (i) The instructions which are translated to machine codes by assembler.

  (ii) The directives that direct the assembler during assembly process, for which no machine code is generated.

1. **ASSUME:** Assume logical segment name.

The ASSUME directive is used to inform the assembler the names of the logical segments to be assumed for different segments used in the program .In the ALP each segment is given name.

Syntax: ASSUME segreg:segname,...segreg:segname

Ex: ASSUME CS:CODE

ASSUME CS:CODE,DS:DATA,SS:STACK

Scanned with CamScanner

2. DB: Define Byte

The DB directive is used to reserve byte or **bytes of memory IOEations in** the available memory.

Syntax: Name of variable DB initialization value.

Ex: MARXS DB 35H,30H,35H,40H

NAME DB "VARDHAMAN"

3. DW: Define Word

The DW directive serves the same puposes as the DB directive,but it now makes the assembler reserve the number of memory words[16-bit) instead of bytes.

Syntax: variable name DW initialization values.

Ex. WORDS DW 1234H,4567H,2367H

WDATA DW 5 Dup(522h)

(or) Dup(?)

4. OD: Define Double:

The directive DD is used to define a double word (4bytes) variable.

Syntax: variablename DD 12345678H

Ex: Data1 DD 12345678H

5. DQ: Define Quad Word

This directive is used to direct the assembler to reserve 4 words (8 bytes) of memory for the specified variable and may initialize it with the specified values.

Syntax: Name of variable DQ initialize values.

Ex. Data1 DQ 123456789ABCDEF2H

6. DT: Define Ten Bytes

The DT directive directs the assemb1er to define the specified variable requiring 10 bytes for its storage and initialize the 10-bytes with the specified values.

Syntax: Name of variable DT initialize values.

Ex: Data1 DT 123456789ABCDEF34567H

7. END: End of Program

The END directive marks the end of an ALP. The statement ner the directive ENO will be ignored by the assembler.

8. ENDP: End of Procedure

The ENDP directive is used to indicate the end of proEedure. In the AL programming the subroutines arB EBIIBd procedures.

Ex: Procedure Start

Start ENDP

Scanned with CamScanner

9. ENDS: End of segment

The ENDS directive is used to indicate the end of segment.

Ex: DATA SEGMENT

DATA ENDS

10.EVEN: Align on Even memory address

The EVEN directives updates the location counter to the next even address.

Ex: EV£N

Procedure Stan

Start ENDP

4 The above structure shows a procedure START that is to be aligned at an even address,

11. EQU: Equate

The directive EQU is used to assign a label with a value or symbol.

Ex: LABEL EQU OSOOH

ADDITION EQU ADD

12. EXTRN: External and public

& The directive EXTRN informs the assembler that the names, procedures and labels declared after this directive have been already defined In some other AL modules.

& While in other module, where names, procedures and labels actually appear, they must be deElared publiE using the PUBLIC directive.

Ex: MODULE1 SEGMENT

PUBLIC FACT FAR

MODULE1 ENDS

MODULE2 SEGMENT

EXTRN FACT FAR

MODULE2 END

13. GROUP: Group the related segments

This directive is used to form logical groups of segments with similar purpose or type.

Ex:PROGRAVGROUPCODE,DATA,STACK

*COOE, DATA and STACX segments lie within a 64KB memory segment that is named as PROGRAM.

14. LABEL: label

The label is used to assign name to the current content of the location counter.

Scanned with CamScanner

Ex: CONTINUE LABEL FAR

The label CONTINUE can be used for a FAR jump, if the program contains the above statement.

15. **LENGTH:** Byte length of a label

This is used to refer to the length of a data array or a string

Ex : MOV CX, LENGTH ARRAY

16. **LOCAL:** The labels, variables, constant or proEedures are declared LOCAL in a module are to be used on1y by the particular module.

Ex : LOCAL a, b, Data1, Array, Routine

17. NAME: logical name of a module

The name directive is used to assign a name to an assembly language program module. The module may now be refer to by its declared name.

Ex : Name "addition"

18. OFFSET: offset of a label

When the assembler comes across the OFFSET operator along with a label, it first computing the 16-bit offset address of a particular label and replace the strlng 'OFFSET LABEL' by the computed offset address.

Ex : MOV Sl, oflset list

19. **ORG:** origin

The ORG directive directs the assembler to start the memory allotment for the particular segment, block OFEode from the declared address in the OhG statement.

Ex: ORG 1000H

20. PROC: Procedure

The PROC directive marks the start of a named procedure in the statement.

Ex: RESULT PROC NEAR

ROUTINE PROC FAR

21. PTR: pointer

The PTR operator is used to declare the type of a label, variable or memory operator.

Ex : MOV AL, BYTE PTR [5I]

MOV BX, WORD PTR [2000H]

22. SEG: segment of a label

The SEG operator is used to decide the segment address of the label, variable or procedure.

Ex : MOV AX, SEG ARRAY

MOV DS, AX

Z3.SEGMENT: logical segment

The segment directive marks the starting of a logical segment

Scanned with CamScanner

Ex: CODE SEGMENT

CODE END5

24.SHORT: The SHORT operator indicates to the assembler that only one byte is required to code the displacement for jump.

Ex : JMP SHDRT LABEL

25.TYPE: The TYPE operator directs the assembler to decide the data type of the specified label and replaces the TYPE label by the deEided data type.

For word variable, the data type is 2.

For double word variable, the data type is 4.

For byte variab1e, the data type is 1.

Ex : STRING DW 2345H, 4567H

MOV AX, TYPE STRING

AX=000ZH

20.GLOBAL: The labels, variables, constants or procedures declared GLOBAL may be used by other modules of the program.

Ex : ROUTINE PROC GLOBAL.

27.FAR PTR: This directive indicates the assembler that the label following FAR PTR is not available within the same segment and the address of the label is of 32-bits i.e 2-bytes of offset followed by 2-bytes of segment address.

Ex : JMP FAR PTR LABEL

28.NEAR PTR: This directive indicates that the label following NEAR PTR is in the same segment and *needs* only 16-bit i.e 2-byte offset to address it

Ex . JMP NEAR PTR LABEL

CALL NEAR PTR ROUTINE

Procedures and Macros•

When we need to use a group of instructions several times throughout a program there are two ways we can avold having to write the group of instructions each time we want to use them.

1. One way is to write the group of instructions as a separate procedure.
2. Another way we can use macros.

Procedures:

- v ThE? proEedure is a group of instructions stored as a separate program in the memory and it ïS EdlIed from the main program whenever required using CALL instruction.
- For calling the procedure we have to store **the return** address (next instruction address followed by CALL) onta the staclC
- At the end of the procedure RET instruction used to return the execution to the next instruction in the main program by retrieving the address from the top of thEz StaEk.

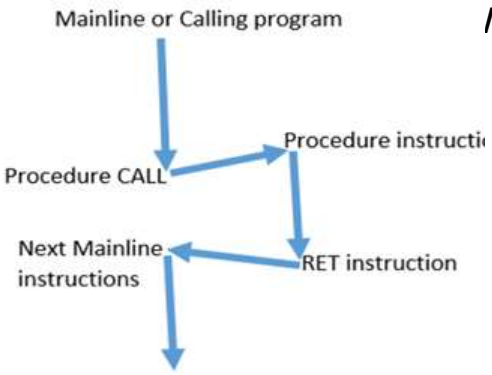
Scanned with CamScanner

Procedures and Macros

When we need to use a group of instructions several times throughout a program there are
two ways, we can avoid having to write the group of instructions each time we want to use them.

Procedure

- To write the group of instructions as a separate procedure.
- We can use macros



Syntax:

Procedure name
PROC near
instructions
RET
Procedure name
ENDP

Macro

A macro is a group of repetitive instructions in a program which are codified only once and can be used as many times as necessary.

Syntax of macro:

macroname
MACRO
instructions
ENDM

Writing Simple 8086 Programs

```
•Example 1: Adding Two Numbers
•; Adding Two Numbers
•;
•; This program adds two numbers stored in registers AX and BX.
•; The result is stored in AX.

•DATA SEGMENT
•   ; No data segment needed for this simple program.
•DATA ENDS

•CODE SEGMENT
•   ASSUME CS:CODE, DS:DATA

•   START:
•       MOV AX, 10  ; Load the first number into AX
•       MOV BX, 20  ; Load the second number into BX

•       ADD AX, BX  ; Add the contents of BX to AX

•       ; The result is now in AX

•   MOV AH, 4CH  ; Function code for exit
•   INT 21H     ; Call the DOS interrupt to terminate the program

•CODE ENDS
•END START
```

- **Example 2: Subtracting Two Numbers**

Code snippet

```
MOV AX, 30
MOV BX, 15
SUB AX, BX
```

- **Example 3: Multiplying Two Numbers**

Code snippet

```
MOV AX, 5
MOV BX, 6
MUL BX
```

- **Example 4: Dividing Two Numbers**

Code snippet

```
MOV AX, 20
MOV BX, 4
DIV BX
```

Assembly Language Program

Development Tools

- **Assemblers:**
  - MASM (Microsoft Macro Assembler)
  - TASM (Turbo Assembler)
- **Linkers:**
  - LINK
- **Debuggers:**
  - DEBUG

Applications of 8086 Programming

1. Embedded Systems:

•Microcontrollers: Many microcontrollers, especially older ones, are based on 8086-like architectures. Knowledge of 8086 assembly language can be useful for programming these devices for specific tasks.

•Real-time Systems: Understanding the timing and synchronization aspects of 8086 programming can be valuable for designing real-time systems.

2. Computer Architecture and Organization:

•Understanding Modern Processors: While modern processors are much more complex, understanding the basics of 8086 architecture can provide a solid foundation for understanding the underlying principles of modern processors.

3. Debugging and Reverse Engineering:

•Analyzing Legacy Systems: Many legacy systems still use 8086-based processors. Understanding 8086 assembly language can be helpful in debugging and reverse engineering these systems.

4. Educational Purposes:

•Teaching Computer Architecture and Assembly Language: 8086 is often used as a teaching tool to introduce students to computer architecture and assembly language programming.

Here are some mini-project ideas that you can explore, incorporating the concepts of 8086 programming:

1. Simple Calculator:

•Create a basic calculator that can perform arithmetic operations like addition, subtraction, multiplication, and division.
•Use 8086 assembly language to implement the logic for these operations.
•Display the results on the screen or through a simple LED display.

2. Number Guessing Game:

•Develop a number guessing game where the computer generates a random number, and the user has to guess it.
•Use 8086 instructions to generate random numbers, compare user input, and provide feedback.
•You can use a simple LCD display or serial communication to display the game's prompts and results.

3. Traffic Light Simulator:

•Simulate a traffic light system using 8086 assembly.
•Use LEDs or a simple display to represent the traffic lights.
•Implement a sequence of red, yellow, and green lights with appropriate time delays.

THANK YOU…